

```
In [29]: import pandas as pd
import seaborn as sns

# Read the Data with Pandas
df = pd.read_csv("train.csv")
df
```

Out[29]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8
...	...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7

891 rows × 12 columns

```
In [31]: # 1. Create df1: Identification and Outcome
# Selecting specifically from the columns you listed
df1 = df[['PassengerId', 'Name', 'Age', 'Survived']].copy()

# 2. Create df2: Socio-Economic and Travel Data
df2 = df[['PassengerId', 'Pclass', 'Sex', 'Fare']].copy()

print("DataFrames successfully created!")
display(df1.head())
display(df2.head())
```

DataFrames successfully created!

	PassengerId	Name	Age	Survived
0	1	Braund, Mr. Owen Harris	22.0	0
1	2	Cumings, Mrs. John Bradley (Florence Briggs Th...	38.0	1
2	3	Heikkinen, Miss. Laina	26.0	1
3	4	Futrelle, Mrs. Jacques Heath (Lily May Peel)	35.0	1
4	5	Allen, Mr. William Henry	35.0	0

	PassengerId	Pclass	Sex	Fare
0	1	3	male	7.2500
1	2	1	female	71.2833
2	3	3	female	7.9250
3	4	1	female	53.1000
4	5	3	male	8.0500

```
In [41]: # Inner Join on 'PassengerId'
merged_inner = pd.merge(df1, df2, on='PassengerId', how='inner')
print("\nInner Join on 'PassengerId':\n")
merged_inner.head()
```

Inner Join on 'PassengerId':

Out[41]:	PassengerId	Name	Age	Survived	Pclass	Sex	Fare
0	1	Braund, Mr. Owen Harris	22.0	0	3	male	7.2500
1	2	Cumings, Mrs. John Bradley (Florence Briggs Th...	38.0	1	1	female	71.2833
2	3	Heikkinen, Miss. Laina	26.0	1	3	female	7.9250
3	4	Futrelle, Mrs. Jacques Heath (Lily May Peel)	35.0	1	1	female	53.1000
4	5	Allen, Mr. William Henry	35.0	0	3	male	8.0500

```
In [43]: # Left Join on 'PassengerId'
merged_left = pd.merge(df1, df2, on='PassengerId', how='left')
print("\nLeft Join on 'PassengerId':\n")
merged_left.head()
```

Left Join on 'PassengerId':

Out[43]:	PassengerId	Name	Age	Survived	Pclass	Sex	Fare
0	1	Braund, Mr. Owen Harris	22.0	0	3	male	7.2500
1	2	Cumings, Mrs. John Bradley (Florence Briggs Th...	38.0	1	1	female	71.2833
2	3	Heikkinen, Miss. Laina	26.0	1	3	female	7.9250
3	4	Futrelle, Mrs. Jacques Heath (Lily May Peel)	35.0	1	1	female	53.1000
4	5	Allen, Mr. William Henry	35.0	0	3	male	8.0500

```
In [45]: # Right Join on 'passengPassengerId_id'
merged_right = pd.merge(df1, df2, on='PassengerId', how='right')
```

```
print("\nRight Join on 'PassengerId':\n")
merged_right.head()
```

Right Join on 'PassengerId':

Out[45]:

	PassengerId	Name	Age	Survived	Pclass	Sex	Fare
0	1	Braund, Mr. Owen Harris	22.0	0	3	male	7.2500
1	2	Cumings, Mrs. John Bradley (Florence Briggs Th...	38.0	1	1	female	71.2833
2	3	Heikkinen, Miss. Laina	26.0	1	3	female	7.9250
3	4	Futrelle, Mrs. Jacques Heath (Lily May Peel)	35.0	1	1	female	53.1000
4	5	Allen, Mr. William Henry	35.0	0	3	male	8.0500

In [47]:

```
# Outer Join on 'passenger_id'
merged_outer = pd.merge(df1, df2, on='PassengerId', how='outer')
print("\nOuter Join on 'PassengerId':\n")
merged_outer.head()
```

Outer Join on 'PassengerId':

Out[47]:

	PassengerId	Name	Age	Survived	Pclass	Sex	Fare
0	1	Braund, Mr. Owen Harris	22.0	0	3	male	7.2500
1	2	Cumings, Mrs. John Bradley (Florence Briggs Th...	38.0	1	1	female	71.2833
2	3	Heikkinen, Miss. Laina	26.0	1	3	female	7.9250
3	4	Futrelle, Mrs. Jacques Heath (Lily May Peel)	35.0	1	1	female	53.1000
4	5	Allen, Mr. William Henry	35.0	0	3	male	8.0500

In [49]:

```
# Concatenate without keys (default behavior)
concatenated = pd.concat([df1, df2])
print("\nConcatenation along rows:\n")
concatenated.head()
```

Concatenation along rows:

Out[49]:

	PassengerId	Name	Age	Survived	Pclass	Sex	Fare
0	1	Braund, Mr. Owen Harris	22.0	0.0	NaN	NaN	NaN
1	2	Cumings, Mrs. John Bradley (Florence Briggs Th...	38.0	1.0	NaN	NaN	NaN
2	3	Heikkinen, Miss. Laina	26.0	1.0	NaN	NaN	NaN
3	4	Futrelle, Mrs. Jacques Heath (Lily May Peel)	35.0	1.0	NaN	NaN	NaN
4	5	Allen, Mr. William Henry	35.0	0.0	NaN	NaN	NaN

In [51]:

```
# Concatenate with specific keys
concatenated_with_keys = pd.concat([df1, df2], keys=['df1', 'df2'])
print("\nConcatenation with keys:\n")
concatenated_with_keys.head()
```

Concatenation with keys:

Out[51]:

	PassengerId	Name	Age	Survived	Pclass	Sex	Fare	
df1	0	1	Braund, Mr. Owen Harris	22.0	0.0	NaN	NaN	NaN
	1	2	Cumings, Mrs. John Bradley (Florence Briggs Th...	38.0	1.0	NaN	NaN	NaN
	2	3	Heikkinen, Miss. Laina	26.0	1.0	NaN	NaN	NaN
	3	4	Futrelle, Mrs. Jacques Heath (Lily May Peel)	35.0	1.0	NaN	NaN	NaN
	4	5	Allen, Mr. William Henry	35.0	0.0	NaN	NaN	NaN

In [53]:

```
# Concatenate along columns (axis=1)
concatenated_axis1 = pd.concat([df1, df2], axis=1)
print("\nConcatenation along columns (axis=1):\n")
concatenated_axis1.head()
```

Concatenation along columns (axis=1):

Out[53]:

	PassengerId	Name	Age	Survived	PassengerId	Pclass	Sex	Fare
0	1	Braund, Mr. Owen Harris	22.0	0	1	3	male	7.2500
1	2	Cumings, Mrs. John Bradley (Florence Briggs Th...	38.0	1	2	1	female	71.2833
2	3	Heikkinen, Miss. Laina	26.0	1	3	3	female	7.9250
3	4	Futrelle, Mrs. Jacques Heath (Lily May Peel)	35.0	1	4	1	female	53.1000
4	5	Allen, Mr. William Henry	35.0	0	5	3	male	8.0500

In [73]:

```
# Calculate mean, median, and mode for numerical columns
print("\n----- Mean ----- \n", titanic[['age', 'fare']].mean())
print("\n----- Median ----- \n", titanic[['age', 'fare']].median())
print("\n----- Mode ----- \n", titanic[['age', 'fare']].mode())
```

```
----- Mean -----
age      29.699118
fare     32.204208
dtype: float64

----- Median -----
age      28.0000
fare     14.4542
dtype: float64

----- Mode -----
age fare
0  24.0  8.05
```

In [67]:

```
# Calculate mid-range (average of max and min for each column)
mid_range = titanic[['age', 'fare']].apply(lambda x: (x.max() + x.min()) / 2)
print("\n----- Mid-Range ----- \n", mid_range)
```

```
----- Mid-Range -----
age      40.2100
fare     256.1646
dtype: float64
```

```
In [2]: import pandas as pd
import numpy as np

# Load Titanic dataset
df = pd.read_csv("train.csv")

# Display the first few rows and info
display(df.head())
df.info()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.250
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.283
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.925
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.100
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.050

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null    int64
1   Survived    891 non-null    int64
2   Pclass      891 non-null    int64
3   Name        891 non-null    object
4   Sex         891 non-null    object
5   Age         714 non-null    float64
6   SibSp       891 non-null    int64
7   Parch       891 non-null    int64
8   Ticket      891 non-null    object
9   Fare        891 non-null    float64
10  Cabin       204 non-null    object
11  Embarked    889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

```
In [10]: # 1. Handling missing values

# Convert to numeric first (results in float64 due to NaNs and decimals)
df['Age'] = pd.to_numeric(df['Age'], errors='coerce')

# To convert to 'Int64', we must first handle the decimals.
# We'll round them to the nearest whole number, then cast.
df['Age'] = df['Age'].round(0).astype('Int64')

# Fill missing 'Embarked' with the most frequent value
```

```
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])
```

```
# Fill missing 'Fare' with the median value  
df['Fare'] = df['Fare'].fillna(df['Fare'].median())
```

```
In [12]: # 2. Convert non-numeric columns to category types to save memory  
df['Pclass'] = df['Pclass'].astype('category')  
df['Survived'] = df['Survived'].astype('category')  
df['Sex'] = df['Sex'].astype('category')  
df['Embarked'] = df['Embarked'].astype('category')
```

```
In [28]: # 3. Check the data types after the conversion and filling missing values  
print("Data types after conversion:")  
df.dtypes
```

Data types after conversion:

```
Out[28]: PassengerId      int64  
Survived      category  
Pclass      category  
Name      object  
Sex      category  
Age      Int64  
SibSp      int64  
Parch      int64  
Ticket      object  
Fare      float64  
Cabin      object  
Embarked      category  
Norm_Fare      float64  
Standardized_Age      Float64  
dtype: object
```

```
In [30]: # 4. Apply Aggregation: Example for 'Age' and 'Fare'  
age_stats = df['Age'].agg(['mean', 'std', 'min', 'max'])  
fare_stats = df['Fare'].agg(['mean', 'std', 'min', 'max'])  
  
# Display the summary statistics  
print("\nAge Stats:")  
age_stats  
print("\nFare Stats:")  
fare_stats
```

Age Stats:

Fare Stats:

```
Out[30]: mean      32.204208  
std      49.693429  
min      0.000000  
max      512.329200  
Name: Fare, dtype: float64
```

```
In [32]: # 5. Apply Normalization (Scaling)  
# Normalize 'Fare' column: scale the 'Fare' to [0, 1] using Min-Max normalization  
min_fare = df['Fare'].min()  
max_fare = df['Fare'].max()  
df['Norm_Fare'] = (df['Fare'] - min_fare) / (max_fare - min_fare)
```

```
In [34]: # 6. Apply Standardization (Z-score scaling)  
# Standardize the 'Age' column: mean = 0, std = 1  
age_mean = df['Age'].mean()  
age_std = df['Age'].std()  
df['Standardized_Age'] = (df['Age'] - age_mean) / age_std
```

```
In [36]: # 7. Display first few rows of the transformed dataframe
```

```
print("\nFirst few rows of the transformed dataframe:")
df.head()
```

First few rows of the transformed dataframe:

```
Out[36]:
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare
0	1	0	3	Braund, Mr. Owen Harris	male	22	1	0	A/5 21171	7.25
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...)	female	38	1	0	PC 17599	71.28
2	3	1	3	Heikkinen, Miss. Laina	female	26	0	0	STON/O2. 3101282	7.92
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35	1	0	113803	53.10
4	5	0	3	Allen, Mr. William Henry	male	35	0	0	373450	8.05

```
In [39]: # 8. Optional: Save the transformed dataframe to a new CSV file
df.to_csv('titanic_transformed.csv', index=False)
```

```
In [40]: import pandas as pd
import numpy as np
```

```
In [42]: # Read the Data with Pandas
csv_data = pd.read_csv("train.csv")
csv_data
```

Out[42]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8
...	...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7

891 rows x 12 columns

```
In [44]: # show only first five rows
csv_data.head()
```


Out [44]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.25
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...)	female	38.0	1	0	PC 17599	71.28
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.92
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.10
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.05

In [46]:

```
# show only first n rows
csv_data.head(10)
```

Out[46]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.25
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...)	female	38.0	1	0	PC 17599	71.28
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.92
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.10
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.05
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330877	8.45
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.86
7	8	0	3	Palsson, Master. Gosta Leonard	male	2.0	3	1	349909	21.01
8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347742	11.13
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14.0	1	0	237736	30.07

In [48]:

```
# show only last five rows
csv_data.tail()
```

Out[48]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.00
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.00
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.45
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.00
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.75

In [50]:

```
# show only last n rows
csv_data.tail(8)
```

Out[50]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare
883	884	0	2	Banfield, Mr. Frederick James	male	28.0	0	0	C.A./SOTON 34068	18.52
884	885	0	3	Sutehall, Mr. Henry Jr	male	25.0	0	0	SOTON/OQ 392076	16.71
885	886	0	3	Rice, Mrs. William (Margaret Norton)	female	39.0	0	5	382652	29.12
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.00
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.00
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.45
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.00
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.75

In [52]:

```
#For negative values of n, this function returns all rows except the first |n| rows
csv_data.tail(-4)
```

Out[52]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	
	4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0
	5	6	0	3	Moran, Mr. James	male	NaN	0	0	330877	8.4
	6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8
	7	8	0	3	Palsson, Master. Gosta Leonard	male	2.0	3	1	349909	21.0
	8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347742	11.1
	...	...	...	...	...	...	...	...	...	...	...
	886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0
	887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0
	888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4
	889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0
	890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7

887 rows × 12 columns

In [54]:

#columns to access the column of the data source
csv_data.columns

Out[54]:

Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
 'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],
 dtype='object')

In [56]:

csv_data.index

Out[56]:

RangeIndex(start=0, stop=891, step=1)

In [58]:

#Return the dtypes in the DataFrame
csv_data.dtypes

```
Out[58]: PassengerId      int64
Survived      int64
Pclass        int64
Name          object
Sex           object
Age           float64
SibSp         int64
Parch         int64
Ticket        object
Fare          float64
Cabin         object
Embarked      object
dtype: object
```

```
In [59]: csv_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   PassengerId     891 non-null    int64
 1   Survived        891 non-null    int64
 2   Pclass          891 non-null    int64
 3   Name            891 non-null    object
 4   Sex             891 non-null    object
 5   Age             714 non-null    float64
 6   SibSp           891 non-null    int64
 7   Parch           891 non-null    int64
 8   Ticket          891 non-null    object
 9   Fare            891 non-null    float64
10   Cabin           204 non-null    object
11   Embarked        889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

```
In [61]: csv_data.values
```

```
Out[61]: array([[1, 0, 3, ..., 7.25, nan, 'S'],
                [2, 1, 1, ..., 71.2833, 'C85', 'C'],
                [3, 1, 3, ..., 7.925, nan, 'S'],
                ...,
                [889, 0, 3, ..., 23.45, nan, 'S'],
                [890, 1, 1, ..., 30.0, 'C148', 'C'],
                [891, 0, 3, ..., 7.75, nan, 'Q']], shape=(891, 12), dtype=object)
```

```
In [63]: csv_data.value_counts()
```

```
Out[63]: PassengerId  Survived  Pclass  Name
Sex      Age      SibSp  Parch  Ticket   Fare      Cabin      Embarked
2                1        1      Cumings, Mrs. John Bradley (Florence Briggs Thayer)
female    38.0    1        0      PC 17599   71.2833   C85        C        1
4                1        1      Futrelle, Mrs. Jacques Heath (Lily May Peel)
female    35.0    1        0      113803    53.1000   C123       S        1
7                0        1      McCarthy, Mr. Timothy J
male      54.0    0        0      17463     51.8625   E46        S        1
11         1        3      Sandstrom, Miss. Marguerite Rut
female    4.0    1        1      PP 9549    16.7000   G6         S        1
12         1        1      Bonnell, Miss. Elizabeth
female    58.0    0        0      113783    26.5500   C103       S        1

..
872         1        1      Beckwith, Mrs. Richard Leonard (Sallie Monypeny)
female    47.0    1        1      11751     52.5542   D35        S        1
873         0        1      Carlsson, Mr. Frans Olof
male      33.0    0        0      695        5.0000   B51 B53 B55 S        1
880         1        1      Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)
female    56.0    0        1      11767     83.1583   C50        C        1
888         1        1      Graham, Miss. Margaret Edith
female    19.0    0        0      112053    30.0000   B42        S        1
890         1        1      Behr, Mr. Karl Howell
male      26.0    0        0      111369    30.0000   C148       C        1
Name: count, Length: 183, dtype: int64
```

```
In [65]: #Return an int representing the number of axes / array dimensions
csv_data.ndim
```

```
Out[65]: 2
```

```
In [67]: #Return an int representing the number of elements in this object
csv_data.size
```

```
Out[67]: 10692
```

```
In [69]: #Return a tuple representing the dimensionality of the DataFrame
csv_data.shape
```

```
Out[69]: (891, 12)
```

```
In [71]: # Indicator whether Series/DataFrame is empty
csv_data.empty
```

```
Out[71]: False
```

```
In [73]: #access column using label
csv_data["PassengerId"]
```

```
Out[73]: 0        1
1        2
2        3
3        4
4        5
...
886     887
887     888
888     889
889     890
890     891
Name: PassengerId, Length: 891, dtype: int64
```

```
In [74]: # Access a group of rows and columns by label(s)
csv_data.loc[5, 'PassengerId']
```

```
Out[74]: np.int64(6)
```

```
In [76]: csv_data.loc[5]
```

Out[76]: PassengerId 6
Survived 0
Pclass 3
Name Moran, Mr. James
Sex male
Age NaN
SibSp 0
Parch 0
Ticket 330877
Fare 8.4583
Cabin NaN
Embarked Q
Name: 5, dtype: object

```
In [78]: csv_data.loc[2:5]
```

Out[78]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.92
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.10
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.05
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330877	8.45

```
In [80]: #Access row  
csv_data.iloc[5]
```

Out[80]: PassengerId 6
Survived 0
Pclass 3
Name Moran, Mr. James
Sex male
Age NaN
SibSp 0
Parch 0
Ticket 330877
Fare 8.4583
Cabin NaN
Embarked Q
Name: 5, dtype: object

```
In [82]: csv_data["Name"].iloc[1]
```

Out[82]: 'Cumings, Mrs. John Bradley (Florence Briggs Thayer)'

```
In [84]: csv_data.iloc[5:10]
```

Out [84]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Far
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330877	8.458
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.862
7	8	0	3	Palsson, Master. Gosta Leonard	male	2.0	3	1	349909	21.075
8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347742	11.133
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14.0	1	0	237736	30.070

In [86]:

```
# Data Filtering
csv_data[csv_data['Sex'] == 'female']
```


Out[86]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53
8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347742	11
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14.0	1	0	237736	30
...	...	...	...	...	...	...	...	...	...	...
880	881	1	2	Shelley, Mrs. William (Imanita Parrish Hall)	female	25.0	0	1	230433	26
882	883	0	3	Dahlberg, Miss. Gerda Ulrika	female	22.0	0	0	7552	10
885	886	0	3	Rice, Mrs. William (Margaret Norton)	female	39.0	0	5	382652	29
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23

314 rows × 12 columns

In [88]:

csv_data[(csv_data.Sex=='female') & (csv_data.Age>60)]

Out[88]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	F
275	276	1	1	Andrews, Miss. Kornelia Theodosia	female	63.0	1	0	13502	77.9
483	484	1	3	Turkula, Mrs. (Hedwig)	female	63.0	0	0	4134	9.5
829	830	1	1	Stone, Mrs. George Nelson (Martha Evelyn)	female	62.0	0	0	113572	80.0

In [89]:

```
# str accessor to filter rows based on strings.
csv_data[csv_data.Sex.str.startswith('m')]
```

Out[89]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330877	8
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51
7	8	0	3	Palsson, Master. Gosta Leonard	male	2.0	3	1	349909	21
...	...	...	...	...	...	...	...	...	...	...
883	884	0	2	Banfield, Mr. Frederick James	male	28.0	0	0	C.A./SOTON 34068	10
884	885	0	3	Sutehall, Mr. Henry Jr	male	25.0	0	0	SOTON/OQ 392076	7
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7

577 rows × 12 columns

In [91]:

```
# isin method to filter the names that exist in a given list
names = ['m','f','male']
csv_data[csv_data.Sex.isin(names)]
```

Out[91]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330877	8
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51
7	8	0	3	Palsson, Master. Gosta Leonard	male	2.0	3	1	349909	21
...	...	...	...	...	...	...	...	...	...	...
883	884	0	2	Banfield, Mr. Frederick James	male	28.0	0	0	C.A./SOTON 34068	10
884	885	0	3	Sutehall, Mr. Henry Jr	male	25.0	0	0	SOTON/OQ 392076	7
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7

577 rows × 12 columns

In [93]:

```
# query function can pass the conditions as a string
csv_data.query('Sex == "female" and Age > 60')
```

Out[93]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	F
275	276	1	1	Andrews, Miss. Kornelia Theodosia	female	63.0	1	0	13502	77.9
483	484	1	3	Turkula, Mrs. (Hedwig)	female	63.0	0	0	4134	9.5
829	830	1	1	Stone, Mrs. George Nelson (Martha Evelyn)	female	62.0	0	0	113572	80.0

In [95]:

```
# Descriptive statistics include those that summarize the central tendency, dispersio
csv_data.describe()
```

Out [95]:

	PassengerId	Survived	Pclass	Age	SibSp	Parch	
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.00
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.20
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.69
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.00
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.91
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.45
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.00
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.32

In [97]:

For numeric data, the result's index will include count, mean, std, min, max as well
For object data (e.g. strings or timestamps), the result's index will include count
The top is the most common value. The freq is the most common value's frequency. Tip
csv_data.describe(include="all")

Out [97]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	
count	891.000000	891.000000	891.000000	891	891	714.000000	891.000000	89
unique	NaN	NaN	NaN	891	2	NaN	NaN	
top	NaN	NaN	NaN	Braund, Mr. Owen Harris	male	NaN	NaN	
freq	NaN	NaN	NaN	1	577	NaN	NaN	
mean	446.000000	0.383838	2.308642	NaN	NaN	29.699118	0.523008	
std	257.353842	0.486592	0.836071	NaN	NaN	14.526497	1.102743	
min	1.000000	0.000000	1.000000	NaN	NaN	0.420000	0.000000	
25%	223.500000	0.000000	2.000000	NaN	NaN	20.125000	0.000000	
50%	446.000000	0.000000	3.000000	NaN	NaN	28.000000	0.000000	
75%	668.500000	1.000000	3.000000	NaN	NaN	38.000000	1.000000	
max	891.000000	1.000000	3.000000	NaN	NaN	80.000000	8.000000	

In [98]:

Including only numeric columns
csv_data.describe(include=[np.number])

Out [98]:

	PassengerId	Survived	Pclass	Age	SibSp	Parch	
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.00
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.20
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.69
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.00
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.91
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.45
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.00
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.32

In [100]:

ignore non-numeric data for processing
csv_data.describe(exclude=["O"])

Out[100...

	PassengerId	Survived	Pclass	Age	SibSp	Parch	
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.00
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.20
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.69
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.00
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.91
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.45
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.00
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.32

In [102...

csv_data.isnull()

Out[102...

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cal
0	False	False	False	False	False	False	False	False	False	False	T
1	False	False	False	False	False	False	False	False	False	False	Fa
2	False	False	False	False	False	False	False	False	False	False	T
3	False	False	False	False	False	False	False	False	False	False	Fa
4	False	False	False	False	False	False	False	False	False	False	T
...	...	...	...	...	...	...	...	...	...	...	
886	False	False	False	False	False	False	False	False	False	False	T
887	False	False	False	False	False	False	False	False	False	False	Fa
888	False	False	False	False	False	True	False	False	False	False	T
889	False	False	False	False	False	False	False	False	False	False	Fa
890	False	False	False	False	False	False	False	False	False	False	T

891 rows × 12 columns

In [104...

csv_data.isna()

Out[104...

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cal
0	False	False	False	False	False	False	False	False	False	False	T
1	False	False	False	False	False	False	False	False	False	False	Fa
2	False	False	False	False	False	False	False	False	False	False	T
3	False	False	False	False	False	False	False	False	False	False	Fa
4	False	False	False	False	False	False	False	False	False	False	T
...	...	...	...	...	...	...	...	...	...	...	
886	False	False	False	False	False	False	False	False	False	False	T
887	False	False	False	False	False	False	False	False	False	False	Fa
888	False	False	False	False	False	True	False	False	False	False	T
889	False	False	False	False	False	False	False	False	False	False	Fa
890	False	False	False	False	False	False	False	False	False	False	T

891 rows × 12 columns

--

```
In [105... # find missing value  
csv_data.isnull().values.any()
```

```
Out[105... np.True_
```

```
In [108... csv_data.isna().values.any()
```

```
Out[108... np.True_
```

```
In [113... csv_data.duplicated().values.any()
```

```
Out[113... np.False_
```

```
In [115... data_missing = pd.read_csv("train.csv")  
print(data_missing.isnull().values.any())
```

```
True
```

```
In [119... data_missing.dropna()
```

Out[119...

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.
10	11	1	3	Sandstrom, Miss. Marguerite Rut	female	4.0	1	1	PP 9549	16.
11	12	1	1	Bonnell, Miss. Elizabeth	female	58.0	0	0	113783	26.
...	...	...	...	...	...	...	...	...	...	
871	872	1	1	Beckwith, Mrs. Richard Leonard (Sallie Monypeny)	female	47.0	1	1	11751	52.
872	873	0	1	Carlsson, Mr. Frans Olof	male	33.0	0	0	695	5.
879	880	1	1	Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)	female	56.0	0	1	11767	83.
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.

183 rows × 12 columns

In [121... data_missing.isnull().values.any()

Out[121... np.True_

In [125... data_missing=data_missing.dropna()
data_missing.isnull().values.any()

Out[125... np.False_

In [127... csv_data= csv_data.drop("Sex", axis=1)
csv_data

Out[127...

	PassengerId	Survived	Pclass	Name	Age	SibSp	Parch	Ticket	Fare	Cabin
	0	1	0	3	Braund, Mr. Owen Harris	22.0	1	0	A/5 21171	7.2500
	1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	38.0	1	0	PC 17599	71.2833
	2	3	1	3	Heikkinen, Miss. Laina	26.0	0	0	STON/O2. 3101282	7.9250
	3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	35.0	1	0	113803	53.1000
	4	5	0	3	Allen, Mr. William Henry	35.0	0	0	373450	8.0500
	...	...	...	...	...	...	...	...	...	...
	886	887	0	2	Montvila, Rev. Juozas	27.0	0	0	211536	13.0000
	887	888	1	1	Graham, Miss. Margaret Edith	19.0	0	0	112053	30.0000
	888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	NaN	1	2	W./C. 6607	23.4500
	889	890	1	1	Behr, Mr. Karl Howell	26.0	0	0	111369	30.0000
	890	891	0	3	Dooley, Mr. Patrick	32.0	0	0	370376	7.7500

891 rows × 11 columns

In [133... csv_data['Survived'].mean()

Out[133... np.float64(0.3838383838383838)

In [137... csv_data['Survived'].median()

Out[137... 0.0

In [141... data_filling=data_missing.fillna(csv_data['Survived'].mean())
data_filling.head(10)

Out[141...

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket		
	1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.
	3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.
	6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.
	10	11	1	3	Sandstrom, Miss. Marguerite Rut	female	4.0	1	1	PP 9549	16.
	11	12	1	1	Bonnell, Miss. Elizabeth	female	58.0	0	0	113783	26.
	21	22	1	2	Beesley, Mr. Lawrence	male	34.0	0	0	248698	13.
	23	24	1	1	Sloper, Mr. William Thompson	male	28.0	0	0	113788	35.
	27	28	0	1	Fortune, Mr. Charles Alexander	male	19.0	3	2	19950	263.
	52	53	1	1	Harper, Mrs. Henry Sleeper (Myna Haxtun)	female	49.0	1	0	PC 17572	76.
	54	55	0	1	Ostby, Mr. Engelhart Cornelius	male	65.0	0	1	113509	61.

In [143...

```
data_filling=data_missing.fillna(csv_data['Survived'].median())
data_filling.head(10)
```

Out[143...

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.
10	11	1	3	Sandstrom, Miss. Marguerite Rut	female	4.0	1	1	PP 9549	16.
11	12	1	1	Bonnell, Miss. Elizabeth	female	58.0	0	0	113783	26.
21	22	1	2	Beesley, Mr. Lawrence	male	34.0	0	0	248698	13.
23	24	1	1	Sloper, Mr. William Thompson	male	28.0	0	0	113788	35.
27	28	0	1	Fortune, Mr. Charles Alexander	male	19.0	3	2	19950	263.
52	53	1	1	Harper, Mrs. Henry Sleeper (Myna Haxtun)	female	49.0	1	0	PC 17572	76.
54	55	0	1	Ostby, Mr. Engelhart Cornelius	male	65.0	0	1	113509	61.

In [155...

```
#In order to convert data types in pandas, there are three basic options:  
#Use astype() to force an appropriate dtype  
#Create a custom function to convert the data  
#Use pandas functions such as to_numeric() or to_datetime()  
csv_data['Age'] = pd.to_numeric(csv_data['Age'], errors='coerce').round().astype('Int64')  
csv_data['Age']
```

Out[155...

022

138

226

335

435

...

88627

88719

888<NA>

88926

89032

Name: Age, Length: 891, dtype: Int64

In [157...

```
csv_data.dtypes
```

```
Out[157... PassengerId      int64
          Survived      int64
          Pclass        int64
          Name          object
          Age           Int64
          SibSp         int64
          Parch         int64
          Ticket        object
          Fare          float64
          Cabin         object
          Embarked      object
dtype: object
```

```
In [22]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from scipy import stats
```

```
In [30]: data = {
    'StudentId': range(1, 31),
    'Name': [
        'Abhishek Podi', 'Abhishek Bachhan', 'Bikash Pandey', 'Anish Joshi', 'Abhay Sharm',
        'Ashish Song', 'Ajay Cinthol', 'Ashutosh Song', 'Ayush Roll17', 'Abhishek Poda',
        'Rohit Kumar', 'Neha Singh', 'Pooja Patil', 'Suresh Mehta', 'Kunal Shah',
        'Aman Verma', 'Riya Desai', 'Vikas Rao', 'Sneha Nair', 'Mohit Jain',
        'Kriti Malhotra', 'Arjun Singh', 'Priya Kapoor', 'Nitin Yadav', 'Shreya Bose',
        'Deepak Gupta', 'Simran Kaur', 'Rahul Mishra', 'Tina Roy', 'Manish Tiwari'
    ],
    'Mathematics': [
        98, 85, 76, 23, np.nan, 100, 125, 55, 21, 88,
        92, 81, 77, 66, 59, 101, 34, 48, 72, 83,
        150, 95, 67, 29, 74, -20, 90, 86, 60, 79
    ],
    'Physics': [
        78, 55, 96, 150, 46, 66, np.nan, 35, 81, 78,
        88, 72, 69, 58, 61, 120, 42, 39, 75, 84,
        160, 91, 70, 28, 73, -10, 89, 87, 62, 80
    ],
    'Chemistry': [
        -95, 45, np.nan, 73, 76, 79, 86, 65, 41, np.nan,
        88, 74, 71, 60, 63, 110, 38, 44, 70, 82,
        145, 90, 68, 25, 72, -30, 91, 85, 59, 77
    ]
}

df = pd.DataFrame(data)
df
```

Out[30]:

	StudentId	Name	Mathematics	Physics	Chemistry
0	1	Abhishek Podi	98.0	78.0	-95.0
1	2	Abhishek Bachhan	85.0	55.0	45.0
2	3	Bikash Pandey	76.0	96.0	NaN
3	4	Anish Joshi	23.0	150.0	73.0
4	5	Abhay Sharma	NaN	46.0	76.0
5	6	Ashish Song	100.0	66.0	79.0
6	7	Ajay Cinthol	125.0	NaN	86.0
7	8	Ashutosh Song	55.0	35.0	65.0
8	9	Ayush Roll17	21.0	81.0	41.0
9	10	Abhishek Poda	88.0	78.0	NaN
10	11	Rohit Kumar	92.0	88.0	88.0
11	12	Neha Singh	81.0	72.0	74.0
12	13	Pooja Patil	77.0	69.0	71.0
13	14	Suresh Mehta	66.0	58.0	60.0
14	15	Kunal Shah	59.0	61.0	63.0
15	16	Aman Verma	101.0	120.0	110.0
16	17	Riya Desai	34.0	42.0	38.0
17	18	Vikas Rao	48.0	39.0	44.0
18	19	Sneha Nair	72.0	75.0	70.0
19	20	Mohit Jain	83.0	84.0	82.0
20	21	Kriti Malhotra	150.0	160.0	145.0
21	22	Arjun Singh	95.0	91.0	90.0
22	23	Priya Kapoor	67.0	70.0	68.0
23	24	Nitin Yadav	29.0	28.0	25.0
24	25	Shreya Bose	74.0	73.0	72.0
25	26	Deepak Gupta	-20.0	-10.0	-30.0
26	27	Simran Kaur	90.0	89.0	91.0
27	28	Rahul Mishra	86.0	87.0	85.0
28	29	Tina Roy	60.0	62.0	59.0
29	30	Manish Tiwari	79.0	80.0	77.0

In [32]:

df.isnull().sum()

Out[32]:

StudentId0
Name0
Mathematics1
Physics1
Chemistry2
dtype: int64

In [34]:

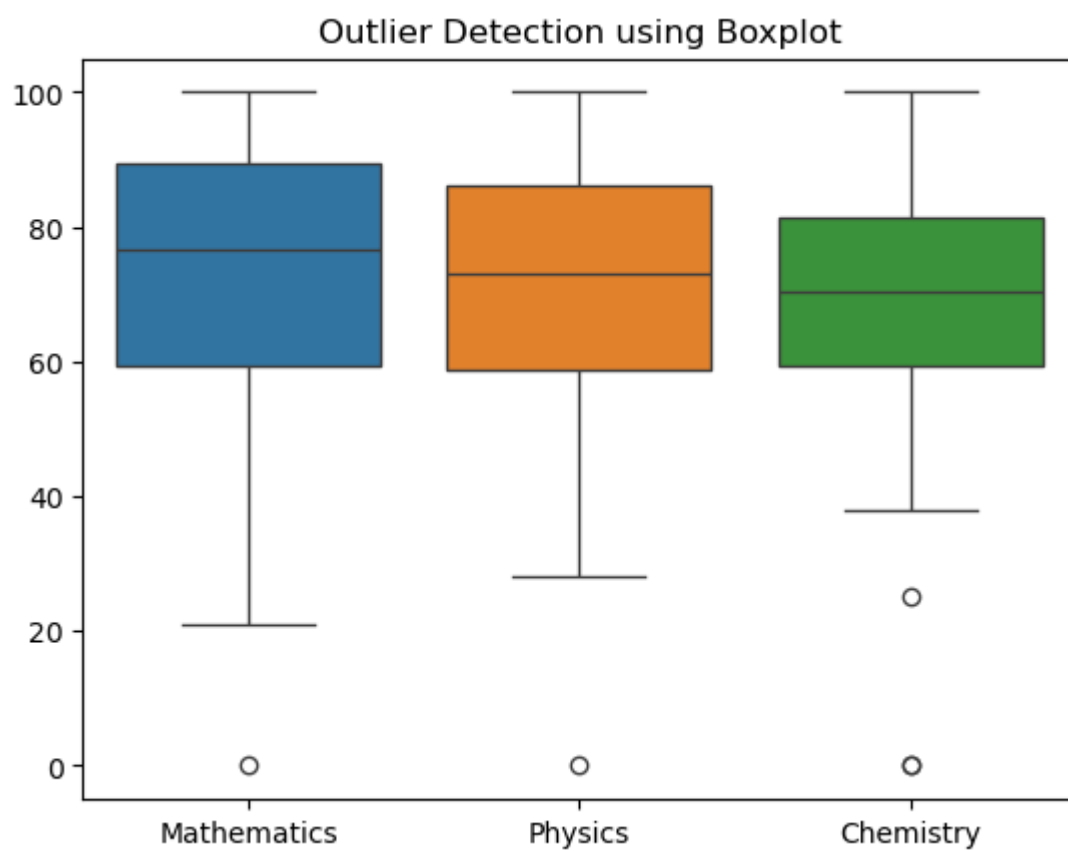
col = ['Mathematics', 'Physics', 'Chemistry']
df[col] = df[col].fillna(df[col].mean())
df

Out[34]:

	StudentId	Name	Mathematics	Physics	Chemistry
0	1	Abhishek Podi	98.000000	78.000000	-95.000000
1	2	Abhishek Bachhan	85.000000	55.000000	45.000000
2	3	Bikash Pandey	76.000000	96.000000	62.571429
3	4	Anish Joshi	23.000000	150.000000	73.000000
4	5	Abhay Sharma	72.206897	46.000000	76.000000
5	6	Ashish Song	100.000000	66.000000	79.000000
6	7	Ajay Cinthol	125.000000	73.206897	86.000000
7	8	Ashutosh Song	55.000000	35.000000	65.000000
8	9	Ayush Roll17	21.000000	81.000000	41.000000
9	10	Abhishek Poda	88.000000	78.000000	62.571429
10	11	Rohit Kumar	92.000000	88.000000	88.000000
11	12	Neha Singh	81.000000	72.000000	74.000000
12	13	Pooja Patil	77.000000	69.000000	71.000000
13	14	Suresh Mehta	66.000000	58.000000	60.000000
14	15	Kunal Shah	59.000000	61.000000	63.000000
15	16	Aman Verma	101.000000	120.000000	110.000000
16	17	Riya Desai	34.000000	42.000000	38.000000
17	18	Vikas Rao	48.000000	39.000000	44.000000
18	19	Sneha Nair	72.000000	75.000000	70.000000
19	20	Mohit Jain	83.000000	84.000000	82.000000
20	21	Kriti Malhotra	150.000000	160.000000	145.000000
21	22	Arjun Singh	95.000000	91.000000	90.000000
22	23	Priya Kapoor	67.000000	70.000000	68.000000
23	24	Nitin Yadav	29.000000	28.000000	25.000000
24	25	Shreya Bose	74.000000	73.000000	72.000000
25	26	Deepak Gupta	-20.000000	-10.000000	-30.000000
26	27	Simran Kaur	90.000000	89.000000	91.000000
27	28	Rahul Mishra	86.000000	87.000000	85.000000
28	29	Tina Roy	60.000000	62.000000	59.000000
29	30	Manish Tiwari	79.000000	80.000000	77.000000

```
In [36]: for col in ['Mathematics', 'Physics', 'Chemistry']:
         df.loc[df[col] > 100, col] = 100
         df.loc[df[col] < 0, col] = 0
```

```
In [38]: sns.boxplot(data=df[['Mathematics', 'Physics', 'Chemistry']])
         plt.title("Outlier Detection using Boxplot")
         plt.show()
```



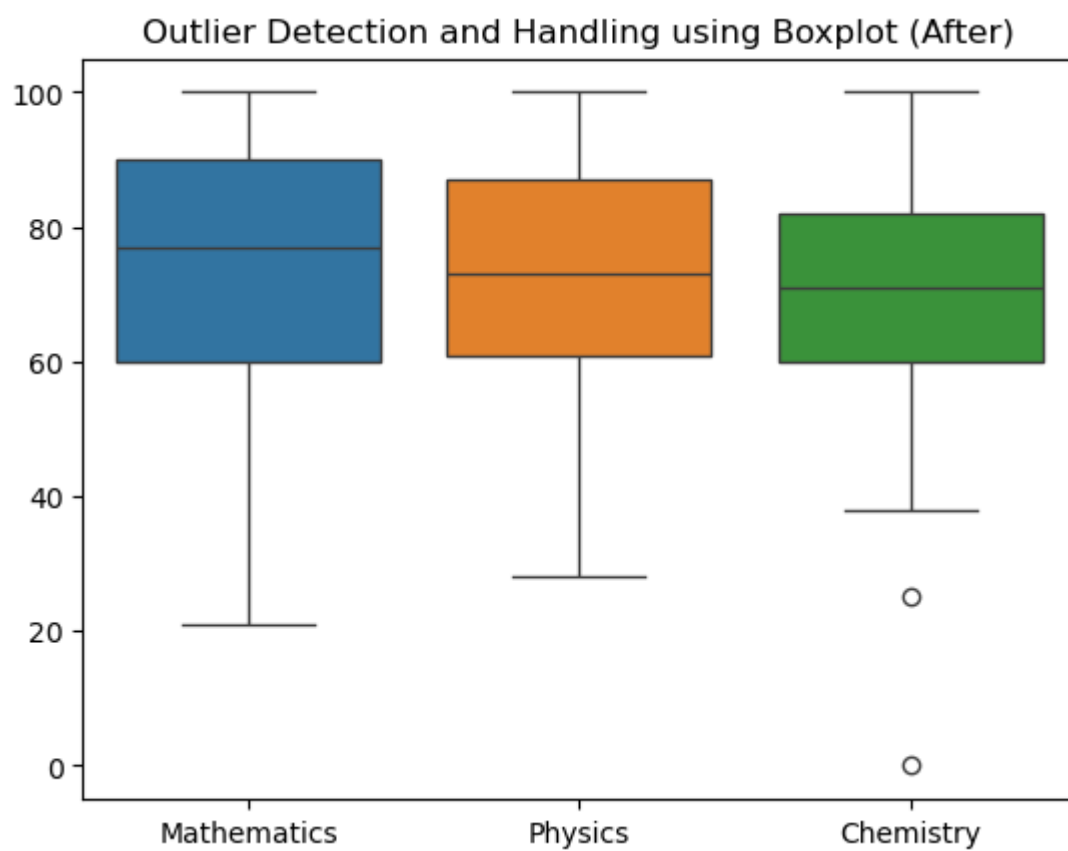
```
In [40]: Q1 = df['Mathematics'].quantile(0.25)
Q3 = df['Mathematics'].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

df['Mathematics'] = np.where(df['Mathematics'] > upper, upper, np.where(df['Mathemati

In [71]: z_scores = np.abs(stats.zscore(df['Physics']))
df = df[z_scores < 2.5]

In [73]: sns.boxplot(data=df[['Mathematics', 'Physics', 'Chemistry']])
plt.title("Outlier Detection and Handling using Boxplot (After)")
plt.show()
```



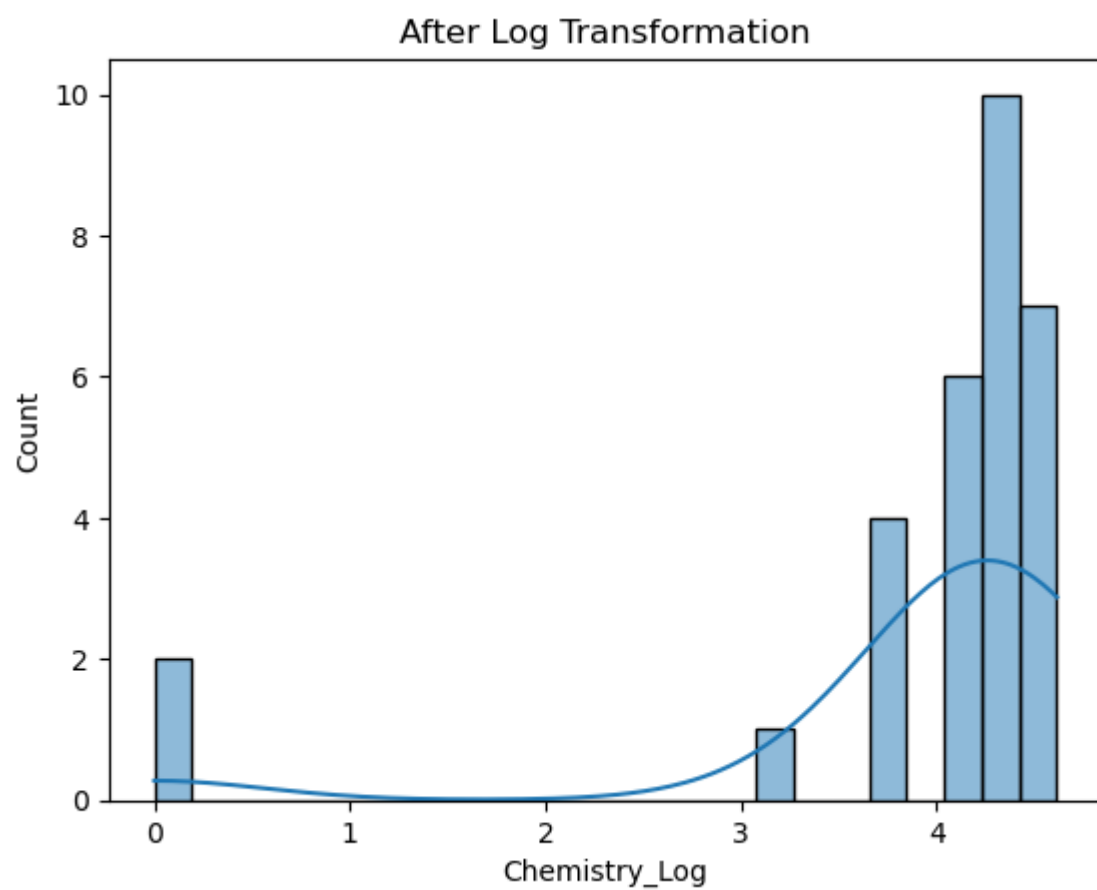
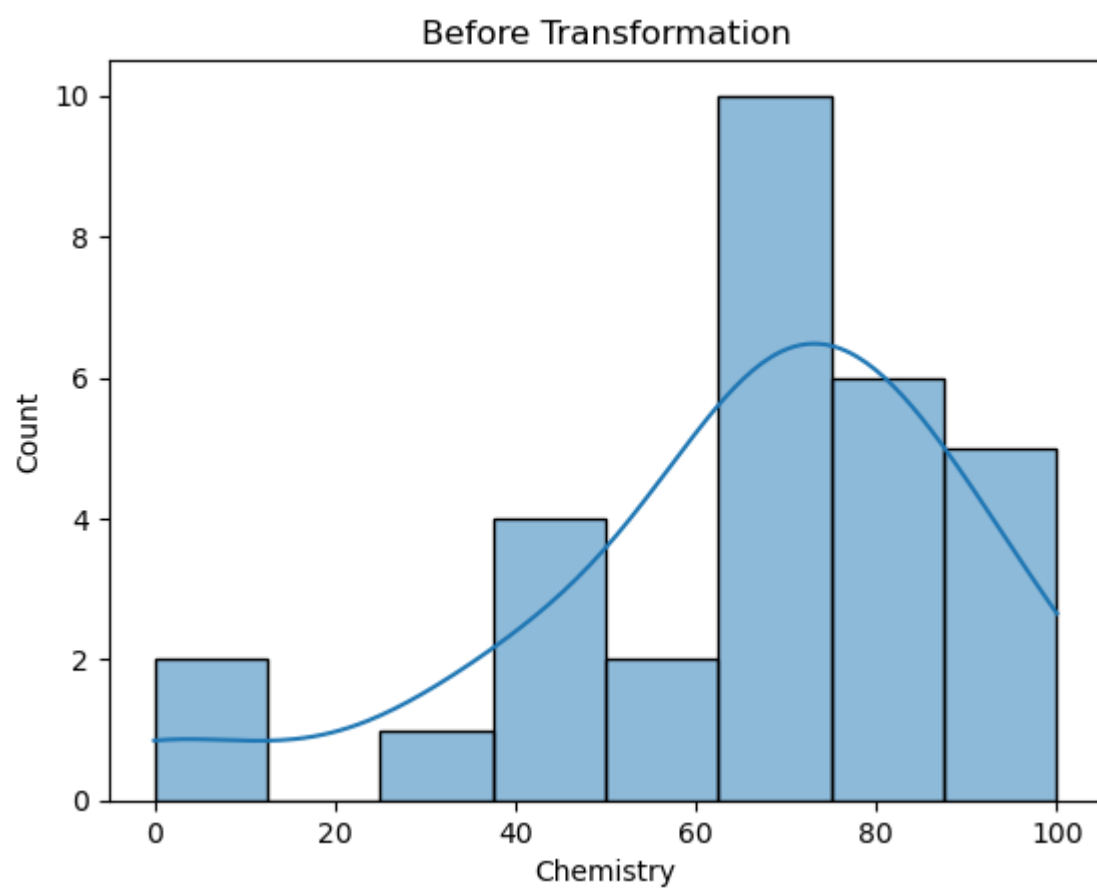
```
In [58]: df['Chemistry'].skew()
```

```
Out[58]: -1.1507378639285284
```

```
In [60]: df['Chemistry_Log'] = np.log(df['Chemistry'] + 1)
```

```
In [62]: sns.histplot(df['Chemistry'], kde=True)
plt.title("Before Transformation")
plt.show()

sns.histplot(df['Chemistry_Log'], kde=True)
plt.title("After Log Transformation")
plt.show()
```

In [64]: df

Out[64]:

	StudentId	Name	Mathematics	Physics	Chemistry	Chemistry_Log
0	1	Abhishek Podi	98.000000	78.000000	0.000000	0.000000
1	2	Abhishek Bachhan	85.000000	55.000000	45.000000	3.828641
2	3	Bikash Pandey	76.000000	96.000000	62.571429	4.152164
3	4	Anish Joshi	23.000000	100.000000	73.000000	4.304065
4	5	Abhay Sharma	72.206897	46.000000	76.000000	4.343805
5	6	Ashish Song	100.000000	66.000000	79.000000	4.382027
6	7	Ajay Cinthol	100.000000	73.206897	86.000000	4.465908
7	8	Ashutosh Song	55.000000	35.000000	65.000000	4.189655
8	9	Ayush Roll17	21.000000	81.000000	41.000000	3.737670
9	10	Abhishek Poda	88.000000	78.000000	62.571429	4.152164
10	11	Rohit Kumar	92.000000	88.000000	88.000000	4.488636
11	12	Neha Singh	81.000000	72.000000	74.000000	4.317488
12	13	Pooja Patil	77.000000	69.000000	71.000000	4.276666
13	14	Suresh Mehta	66.000000	58.000000	60.000000	4.110874
14	15	Kunal Shah	59.000000	61.000000	63.000000	4.158883
15	16	Aman Verma	100.000000	100.000000	100.000000	4.615121
16	17	Riya Desai	34.000000	42.000000	38.000000	3.663562
17	18	Vikas Rao	48.000000	39.000000	44.000000	3.806662
18	19	Sneha Nair	72.000000	75.000000	70.000000	4.262680
19	20	Mohit Jain	83.000000	84.000000	82.000000	4.418841
20	21	Kriti Malhotra	100.000000	100.000000	100.000000	4.615121
21	22	Arjun Singh	95.000000	91.000000	90.000000	4.510860
22	23	Priya Kapoor	67.000000	70.000000	68.000000	4.234107
23	24	Nitin Yadav	29.000000	28.000000	25.000000	3.258097
24	25	Shreya Bose	74.000000	73.000000	72.000000	4.290459
25	26	Deepak Gupta	13.875000	0.000000	0.000000	0.000000
26	27	Simran Kaur	90.000000	89.000000	91.000000	4.521789
27	28	Rahul Mishra	86.000000	87.000000	85.000000	4.454347
28	29	Tina Roy	60.000000	62.000000	59.000000	4.094345
29	30	Manish Tiwari	79.000000	80.000000	77.000000	4.356709

In []:

In []:

In []:

In []:

In []:

In []:


```
In [17]: import pandas as pd
import numpy as np

df = pd.read_csv("adult.csv")
df
```

Out[17]:

	age	workclass	fnlwgt	education	education.num	marital.status	occupation
0	90	?	77053	HS-grad	9	Widowed	?
1	82	Private	132870	HS-grad	9	Widowed	Exec-managerial
2	66	?	186061	Some-college	10	Widowed	?
3	54	Private	140359	7th-8th	4	Divorced	Machine-op-inspct
4	41	Private	264663	Some-college	10	Separated	Prof-specialty
...	...	...	...	...	...	...	...
32556	22	Private	310152	Some-college	10	Never-married	Protective-serv
32557	27	Private	257302	Assoc-acdm	12	Married-civ-spouse	Tech-support
32558	40	Private	154374	HS-grad	9	Married-civ-spouse	Machine-op-inspct
32559	58	Private	151910	HS-grad	9	Widowed	Adm-clerical
32560	22	Private	201490	HS-grad	9	Never-married	Adm-clerical

32561 rows × 15 columns

```
In [19]: df = df[["education", "age"]]
age_list_by_education = df.groupby("education")["age"].apply(list)
print("List of Age values grouped by Education:\n")
age_list_by_education
```

List of Age values grouped by Education:

Out[19]:

education	
10th	[38, 37, 38, 59, 28, 47, 39, 35, 27, 51, 58, 5...
11th	[45, 29, 50, 64, 19, 35, 43, 32, 48, 31, 27, 4...
12th	[33, 39, 25, 17, 56, 19, 20, 43, 79, 39, 19, 6...
1st-4th	[33, 58, 73, 57, 53, 61, 48, 32, 67, 68, 64, 5...
5th-6th	[49, 46, 61, 69, 47, 43, 46, 56, 61, 26, 41, 3...
7th-8th	[54, 42, 28, 41, 29, 56, 50, 24, 48, 40, 48, 6...
9th	[49, 45, 24, 47, 17, 19, 60, 39, 43, 55, 45, 6...
Assoc-acdm	[22, 43, 40, 46, 20, 45, 34, 32, 28, 30, 25, 2...
Assoc-voc	[21, 51, 33, 60, 41, 39, 26, 43, 41, 68, 29, 2...
Bachelors	[52, 34, 37, 51, 44, 48, 32, 38, 51, 39, 49, 5...
Doctorate	[74, 45, 51, 63, 37, 59, 46, 72, 72, 68, 66, 4...
HS-grad	[90, 82, 34, 68, 61, 61, 60, 53, 71, 71, 73, 5...
Masters	[32, 57, 54, 43, 43, 39, 37, 39, 46, 40, 31, 4...
Preschool	[26, 40, 27, 32, 53, 51, 71, 31, 33, 39, 52, 5...
Prof-school	[38, 46, 52, 51, 40, 41, 55, 42, 41, 60, 39, 4...
Some-college	[66, 41, 41, 51, 23, 51, 68, 67, 42, 53, 40, 4...
Name: age, dtype: object	

```
In [21]: summary_statistics = df.groupby("education")["age"].agg(
    Mean="mean",
    Median="median",
```

```

        Minimum="min",
        Maximum="max",
        Standard_Deviation="std"
    )

    print("\nSummary Statistics of Age grouped by Education:\n")
    summary_statistics

```

Summary Statistics of Age grouped by Education:

Out[21]:

	Mean	Median	Minimum	Maximum	Standard_Deviation
education					
10th	37.429796	34.0	17	90	16.720713
11th	32.355745	28.0	17	90	15.545485
12th	32.000000	28.0	17	79	14.334625
1st-4th	46.142857	46.0	19	90	15.615625
5th-6th	42.885886	42.0	17	84	15.557285
7th-8th	48.445820	50.0	17	90	16.092350
9th	41.060311	39.0	17	90	15.946862
Assoc-acdm	37.381443	36.0	19	90	11.095177
Assoc-voc	38.553546	37.0	19	84	11.631300
Bachelors	38.904949	37.0	19	90	11.912210
Doctorate	47.702179	47.0	24	80	11.784716
HS-grad	38.974479	37.0	17	90	13.541524
Masters	44.049913	43.0	18	90	11.068935
Preschool	42.764706	41.0	19	75	15.126914
Prof-school	44.746528	43.0	25	90	11.962477
Some-college	35.756275	34.0	17	90	13.474051

In [23]:

```

df = pd.read_csv("Iris.csv")
df

```

Out[23]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
	0	1	5.1	3.5	1.4	Iris-setosa
	1	2	4.9	3.0	1.4	Iris-setosa
	2	3	4.7	3.2	1.3	Iris-setosa
	3	4	4.6	3.1	1.5	Iris-setosa
	4	5	5.0	3.6	1.4	Iris-setosa
	...	...	...	...	...	...
	145	146	6.7	3.0	5.2	Iris-virginica
	146	147	6.3	2.5	5.0	Iris-virginica
	147	148	6.5	3.0	5.2	Iris-virginica
	148	149	6.2	3.4	5.4	Iris-virginica
	149	150	5.9	3.0	5.1	Iris-virginica

150 rows × 6 columns

In [27]:

```
df = df.drop(columns=["Id"], errors="ignore")

species_names = ["Iris-setosa", "Iris-versicolor", "Iris-virginica"]

for species in species_names:
    print(f"\nStatistical Details for {species}\n")

    species_data = df[df["Species"] == species].drop(columns=["Species"])

    # Mean
    print("Mean:")
    print(species_data.mean())

    # Standard Deviation
    print("\nStandard Deviation:")
    print(species_data.std())

    # Percentiles
    print("\nPercentiles (25%, 50%, 75%):")
    print(np.percentile(species_data, [25, 50, 75], axis=0))
```

Statistical Details for Iris-setosa

Mean:

```
SepalLengthCm    5.006
SepalWidthCm      3.418
PetalLengthCm     1.464
PetalWidthCm      0.244
dtype: float64
```

Standard Deviation:

```
SepalLengthCm    0.352490
SepalWidthCm      0.381024
PetalLengthCm     0.173511
PetalWidthCm      0.107210
dtype: float64
```

Percentiles (25%, 50%, 75%):

```
[[4.8   3.125 1.4   0.2 ]
 [5.    3.4   1.5   0.2 ]
 [5.2   3.675 1.575 0.3  ]]
```

Statistical Details for Iris-versicolor

Mean:

```
SepalLengthCm    5.936
SepalWidthCm      2.770
PetalLengthCm     4.260
PetalWidthCm      1.326
dtype: float64
```

Standard Deviation:

```
SepalLengthCm    0.516171
SepalWidthCm      0.313798
PetalLengthCm     0.469911
PetalWidthCm      0.197753
dtype: float64
```

Percentiles (25%, 50%, 75%):

```
[[5.6   2.525 4.    1.2 ]
 [5.9   2.8   4.35  1.3 ]
 [6.3   3.    4.6   1.5  ]]
```

Statistical Details for Iris-virginica

Mean:

```
SepalLengthCm    6.588
SepalWidthCm      2.974
PetalLengthCm     5.552
PetalWidthCm      2.026
dtype: float64
```

Standard Deviation:

```
SepalLengthCm    0.635880
SepalWidthCm      0.322497
PetalLengthCm     0.551895
PetalWidthCm      0.274650
dtype: float64
```

Percentiles (25%, 50%, 75%):

```
[[6.225 2.8   5.1   1.8 ]
 [6.5   3.    5.55  2.   ]
 [6.9   3.175 5.875 2.3  ]]
```

```
In [46]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```
In [48]: data = pd.read_csv("BostonHousing.csv")
data
```

Out[48]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO		
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1	296	15.3	396.9	
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2	242	17.8	396.9	
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2	242	17.8	392.8	
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3	222	18.7	394.6	
4	0.06905	0.0	2.18	0.0	0.458	7.147	54.2	6.0622	3	222	18.7	396.9	
...	...	...	...	...	...	...	...	...	...	...	...	...	.
501	0.06263	0.0	11.93	0.0	0.573	6.593	69.1	2.4786	1	273	21.0	391.9	
502	0.04527	0.0	11.93	0.0	0.573	6.120	76.7	2.2875	1	273	21.0	396.9	
503	0.06076	0.0	11.93	0.0	0.573	6.976	91.0	2.1675	1	273	21.0	396.9	
504	0.10959	0.0	11.93	0.0	0.573	6.794	89.3	2.3889	1	273	21.0	393.4	
505	0.04741	0.0	11.93	0.0	0.573	6.030	NaN	2.5050	1	273	21.0	396.9	

506 rows × 14 columns

```
In [50]: print(data.isnull().sum())

CRIM      20
ZN        20
INDUS     20
CHAS      20
NOX        0
RM         0
AGE       20
DIS        0
RAD        0
TAX        0
PTRATIO    0
B          0
LSTAT     20
MEDV       0
dtype: int64
```

```
In [52]: data.fillna(data.mean(), inplace=True)
```

```
In [54]: print(data.isnull().sum())
```



```
CRIM      0
ZN         0
INDUS     0
CHAS      0
NOX       0
RM        0
AGE       0
DIS       0
RAD       0
TAX       0
PTRATIO   0
B         0
LSTAT     0
MEDV      0
dtype: int64
```

```
In [56]: X = data[['RM']]      # Single independent variable
        y = data['MEDV']      # Target variable
```

```
In [58]: X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
        )
```

```
In [60]: single_model = LinearRegression()
        single_model.fit(X_train, y_train)
```

```
Out[60]: ▼ LinearRegression ⓘ ?
        ► Parameters
```

```
In [62]: print("Intercept:", single_model.intercept_)
        print("Coefficient:", single_model.coef_[0])
```

```
Intercept: -36.24631889813795
Coefficient: 9.348301406497727
```

```
In [64]: y_pred_single = single_model.predict(X_test)

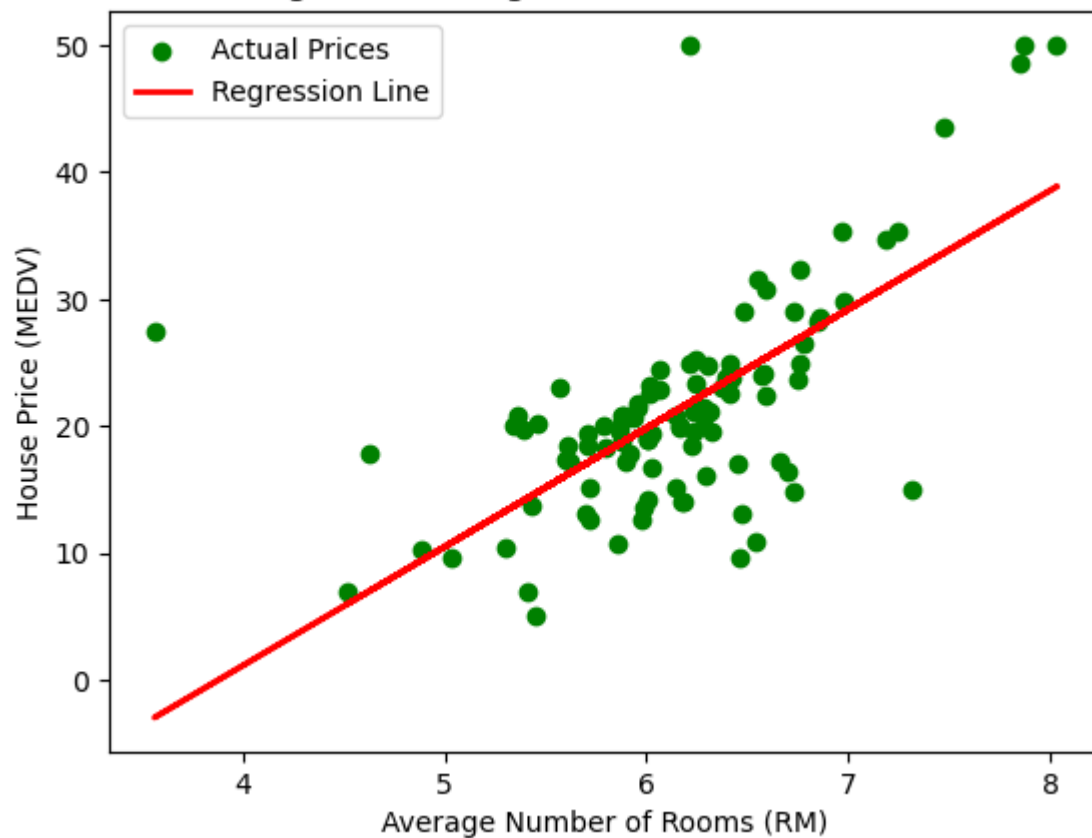
        mse_single = mean_squared_error(y_test, y_pred_single)
        r2_single = r2_score(y_test, y_pred_single)

        print("Mean Squared Error:", mse_single)
        print("R² Score:", r2_single)
```

```
Mean Squared Error: 46.144775347317264
R² Score: 0.3707569232254778
```

```
In [66]: plt.scatter(X_test, y_test, color='green', label='Actual Prices')
        plt.plot(X_test, y_pred_single, color='red', linewidth=2, label='Regression Line')
        plt.xlabel("Average Number of Rooms (RM)")
        plt.ylabel("House Price (MEDV)")
        plt.title("Single Linear Regression: RM vs House Price")
        plt.legend()
        plt.show()
```

Single Linear Regression: RM vs House Price



In []:

```
In [69]: X = data.drop('MEDV', axis=1)
y = data['MEDV']
```

```
In [71]: X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
In [73]: multi_model = LinearRegression()
multi_model.fit(X_train, y_train)
```

```
Out[73]: ▼ LinearRegression ⓘ ?
          ► Parameters
```

```
In [75]: coefficients = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': multi_model.coef_
})

print(coefficients)
```

	Feature	Coefficient
0	CRIM	-0.114140
1	ZN	0.029016
2	INDUS	-0.027133
3	CHAS	3.240673
4	NOX	-16.017543
5	RM	4.752410
6	AGE	-0.019520
7	DIS	-1.521593
8	RAD	0.222823
9	TAX	-0.009151
10	PTRATIO	-0.886045
11	B	0.012263
12	LSTAT	-0.438440

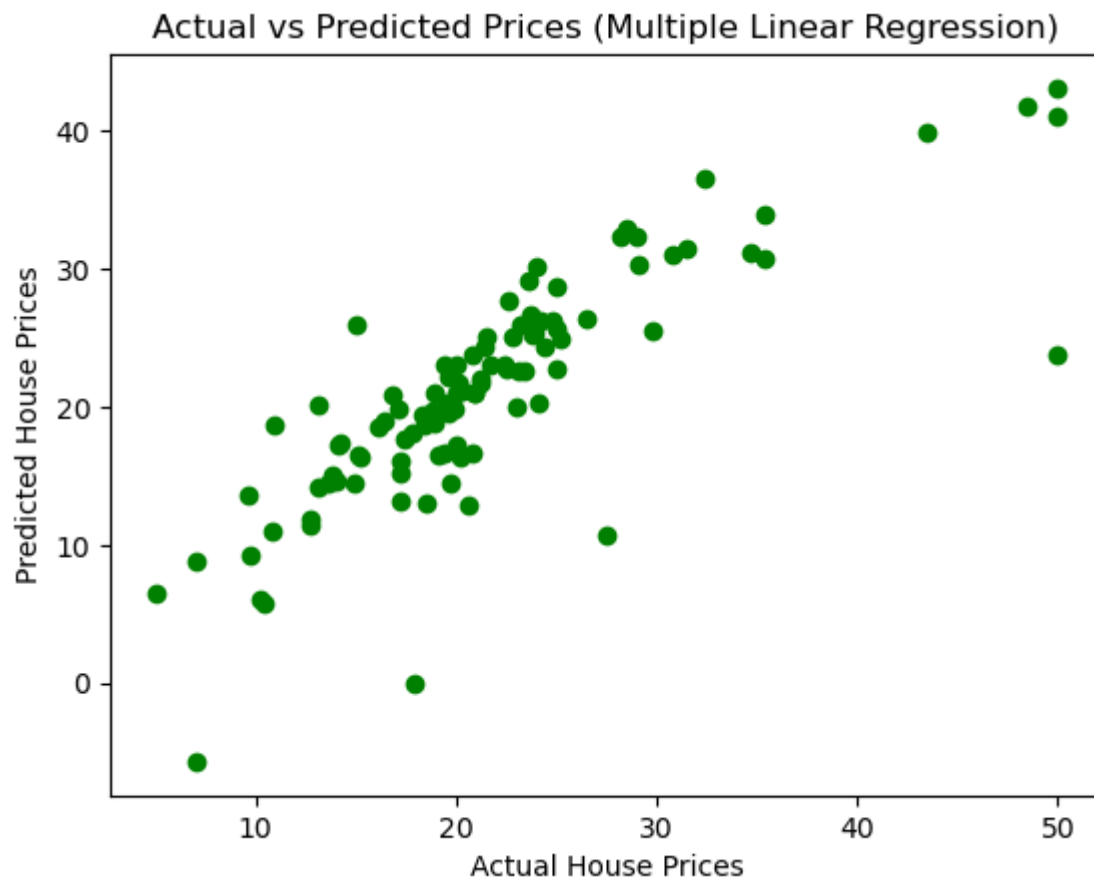
```
In [77]: y_pred_multi = multi_model.predict(X_test)

mse_multi = mean_squared_error(y_test, y_pred_multi)
r2_multi = r2_score(y_test, y_pred_multi)

print("Mean Squared Error:", mse_multi)
print("R2 Score:", r2_multi)
```

Mean Squared Error: 25.017672023842582
R² Score: 0.6588520195508156

```
In [79]: plt.scatter(y_test, y_pred_multi, color='green')
plt.xlabel("Actual House Prices")
plt.ylabel("Predicted House Prices")
plt.title("Actual vs Predicted Prices (Multiple Linear Regression)")
plt.show()
```



In []:

```
In [193... import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [195... dataset = pd.read_csv("Social_Network_Ads.csv")
dataset.head()
```

```
Out[195...      User ID  Gender  Age  EstimatedSalary  Purchased
0    15624510   Male   19           19000           0
1    15810944   Male   35           20000           0
2    15668575  Female   26           43000           0
3    15603246  Female   27           57000           0
4    15804002   Male   19           76000           0
```

```
In [197... dataset.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 400 entries, 0 to 399
Data columns (total 5 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   User ID               400 non-null   int64  
 1   Gender                400 non-null   object  
 2   Age                   400 non-null   int64  
 3   EstimatedSalary       400 non-null   int64  
 4   Purchased             400 non-null   int64  
dtypes: int64(4), object(1)
memory usage: 15.8+ KB
```

```
In [199... X = dataset.iloc[:, [2, 3]].values   # Age and Estimated Salary
y = dataset.iloc[:, 4].values        # Purchased
```

```
In [201... #MODEL 1 : WITH random_state
```

```
In [202... from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=0
)
```

```
In [204... from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
In [206... from sklearn.linear_model import LogisticRegression

classifier = LogisticRegression(random_state=0)
classifier.fit(X_train, y_train)
```

```
Out[206... ▼ LogisticRegression ⓘ ?
    ► Parameters
```

```
In [208... y_pred = classifier.predict(X_test)
```

y_pred

```
Out[208... array([0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1,
        0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,
        1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1,
        0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1,
        0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1])
```

```
In [210... from sklearn.metrics import confusion_matrix
```

```
cm = confusion_matrix(y_test, y_pred)
cm
```

```
Out[210... array([[65,  3],
        [ 8, 24]])
```

```
In [212... from sklearn.metrics import confusion_matrix
```

```
confusion_matrix_rs = confusion_matrix(y_test, y_pred)
print("Confusion Matrix (WITH random_state):\n", confusion_matrix_rs)
```

```
TN = confusion_matrix_rs[0][0]
FP = confusion_matrix_rs[0][1]
FN = confusion_matrix_rs[1][0]
TP = confusion_matrix_rs[1][1]
```

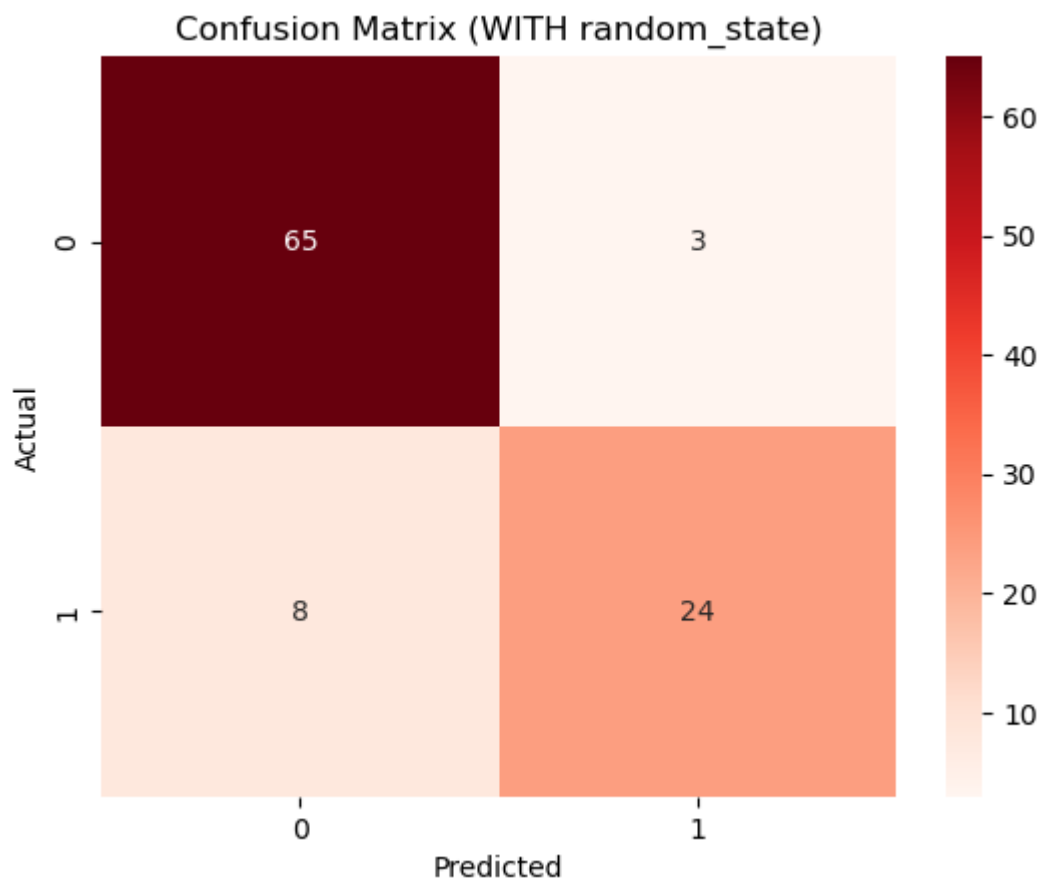
```
accuracy = (TP + TN) / (TP + TN + FP + FN)
error_rate = 1 - accuracy
precision = TP / (TP + FP)
recall = TP / (TP + FN)
```

```
print("\nWITH random_state")
print("Accuracy:", accuracy)
print("Error Rate:", error_rate)
print("Precision:", precision)
print("Recall:", recall)
```

```
Confusion Matrix (WITH random_state):
[[65  3]
 [ 8 24]]
```

```
WITH random_state
Accuracy: 0.89
Error Rate: 0.10999999999999999
Precision: 0.8888888888888888
Recall: 0.75
```

```
In [214... sns.heatmap(confusion_matrix_rs, annot=True, fmt='d', cmap='Reds')
plt.title("Confusion Matrix (WITH random_state)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



```
In [216... from matplotlib.colors import ListedColormap

X_set, y_set = X_train, y_train

X1, X2 = np.meshgrid(
    np.arange(X_set[:, 0].min() - 1, X_set[:, 0].max() + 1, 0.01),
    np.arange(X_set[:, 1].min() - 1, X_set[:, 1].max() + 1, 0.01)
)

plt.contourf(
    X1, X2,
    classifier.predict(np.array([X1.ravel(), X2.ravel()]).T).reshape(X1.shape),
    alpha=0.75,
    cmap=ListedColormap(('black', 'red'))
)

plt.scatter(
    X_set[:, 0], X_set[:, 1],
    c=y_set,
    cmap=ListedColormap(('black', 'red'))
)

plt.title("Logistic Regression (WITH random_state)")
plt.xlabel("Age")
plt.ylabel("Estimated Salary")
plt.show()
```



In [217...] *#MODEL 2 : WITHOUT random_state*

```
In [219...] X_train2, X_test2, y_train2, y_test2 = train_test_split(
            X, y, test_size=0.25
        )
```

```
In [220...] scaler2 = StandardScaler()
X_train2 = scaler2.fit_transform(X_train2)
X_test2 = scaler2.transform(X_test2)
```

```
In [223...] classifier2 = LogisticRegression()
classifier2.fit(X_train2, y_train2)
```

Out[223...] **LogisticRegression** ⓘ ?

► Parameters

```
In [226...] y_pred2 = classifier2.predict(X_test2)
y_pred2
```

```
Out[226...] array([1, 1, 0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0,
        0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0,
        1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0,
        1, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0,
        1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0])
```

```
In [227...] confusion_matrix_no_rs = confusion_matrix(y_test2, y_pred2)
print("\nConfusion Matrix (WITHOUT random_state):\n", confusion_matrix_no_rs)

TN2 = confusion_matrix_no_rs[0][0]
FP2 = confusion_matrix_no_rs[0][1]
FN2 = confusion_matrix_no_rs[1][0]
TP2 = confusion_matrix_no_rs[1][1]

accuracy2 = (TP2 + TN2) / (TP2 + TN2 + FP2 + FN2)
```

```

error_rate2 = 1 - accuracy2
precision2 = TP2 / (TP2 + FP2)
recall2 = TP2 / (TP2 + FN2)

print("\nWITHOUT random_state")
print("Accuracy:", accuracy2)
print("Error Rate:", error_rate2)
print("Precision:", precision2)
print("Recall:", recall2)

```

Confusion Matrix (WITHOUT random_state):

```

[[62  8]
 [10 20]]

```

WITHOUT random_state

Accuracy: 0.82

Error Rate: 0.180000000000000005

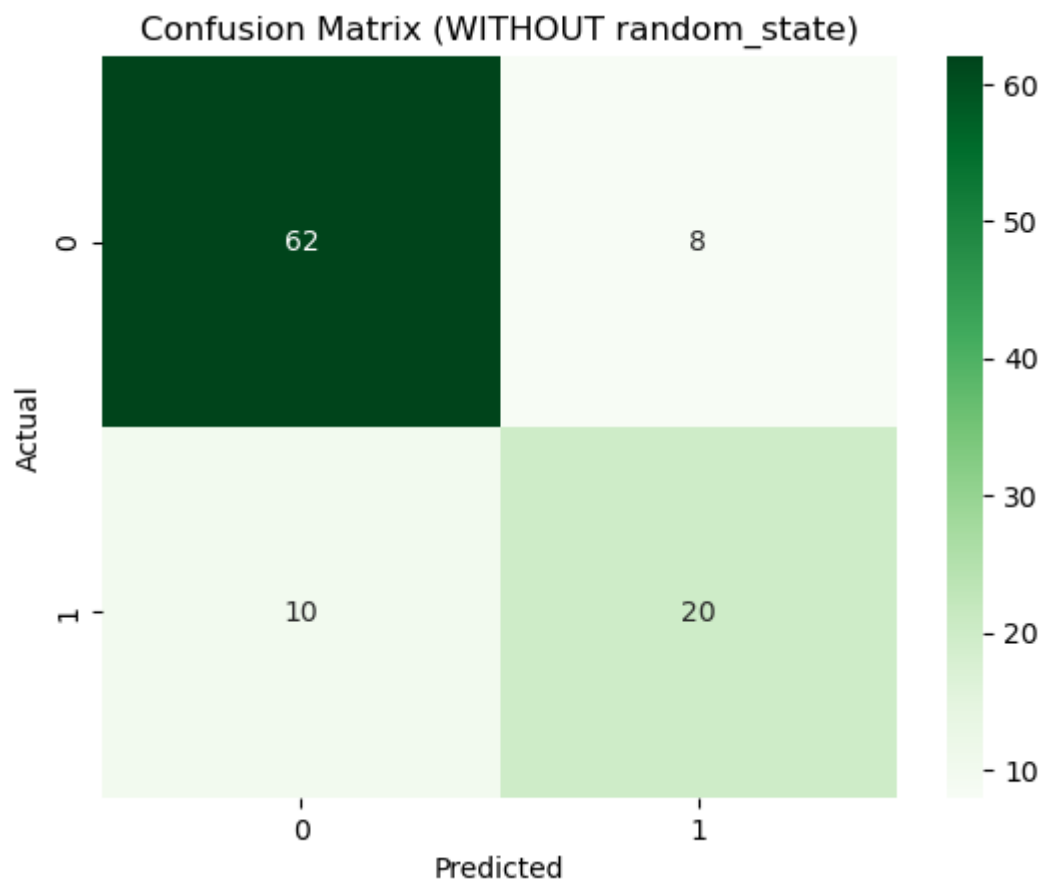
Precision: 0.7142857142857143

Recall: 0.6666666666666666

```

In [229...] sns.heatmap(confusion_matrix_no_rs, annot=True, fmt='d', cmap='Greens')
plt.title("Confusion Matrix (WITHOUT random_state)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

```



```

In [230...] X_set, y_set = X_train2, y_train2

X1, X2 = np.meshgrid(
    np.arange(X_set[:, 0].min() - 1, X_set[:, 0].max() + 1, 0.01),
    np.arange(X_set[:, 1].min() - 1, X_set[:, 1].max() + 1, 0.01)
)

plt.contourf(
    X1, X2,
    classifier2.predict(np.array([X1.ravel(), X2.ravel()]).T).reshape(X1.shape),
    alpha=0.75,
    cmap=ListedColormap(['black', 'green'])
)

```



```
plt.scatter(  
    X_set[:, 0], X_set[:, 1],  
    c=y_set,  
    cmap=ListedColormap(('black', 'green'))  
)  
  
plt.title("Logistic Regression (WITHOUT random_state)")  
plt.xlabel("Age")  
plt.ylabel("Estimated Salary")  
plt.show()
```



In []:

```
In [100... import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap

from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
```

```
In [102... df = pd.read_csv("Iris.csv")

if 'Id' in df.columns:
    df = df.drop(columns=['Id'])

df.head()
```

```
Out[102...      SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm  Species
0                5.1             3.5            1.4           0.2  Iris-setosa
1                4.9             3.0            1.4           0.2  Iris-setosa
2                4.7             3.2            1.3           0.2  Iris-setosa
3                4.6             3.1            1.5           0.2  Iris-setosa
4                5.0             3.6            1.4           0.2  Iris-setosa
```

```
In [104... X = df.iloc[:, [0,1]].values  # SepalLength & SepalWidth
y = df.iloc[:, -1].values
```

```
In [106... from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
y = le.fit_transform(y)
```

```
In [108... X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=0
)
```

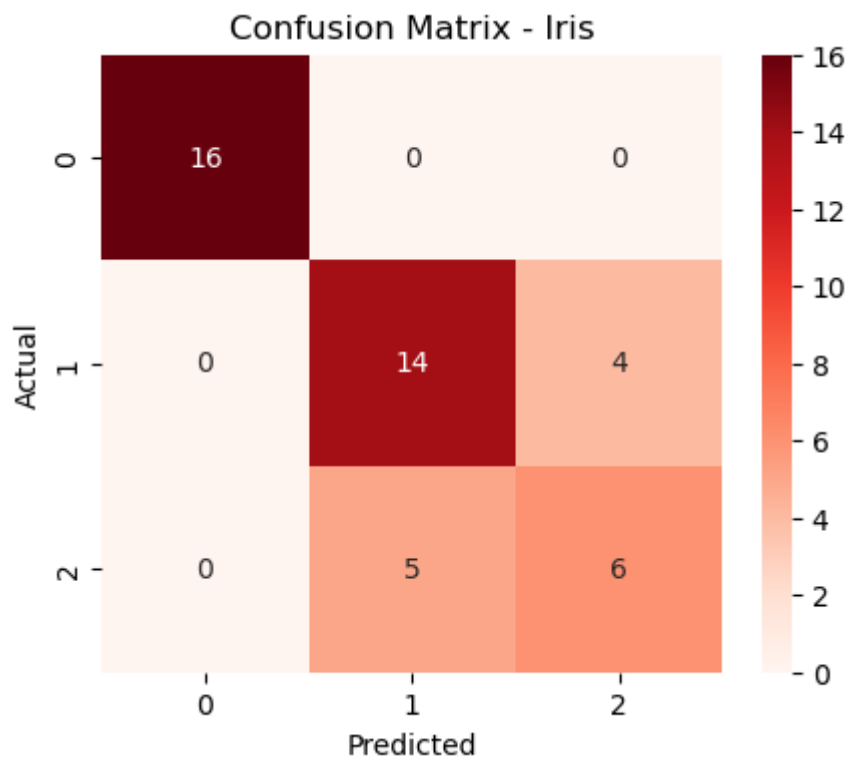
```
In [110... model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

```
In [112... cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
```

```
Confusion Matrix:
[[16  0  0]
 [ 0 14  4]
 [ 0  5  6]]
```

```
In [114... plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Reds')
plt.title("Confusion Matrix - Iris")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



```
In [116... for i in range(len(cm)):  
    TP = cm[i, i]  
    FN = sum(cm[i, :]) - TP  
    FP = sum(cm[:, i]) - TP  
    TN = sum(sum(cm)) - (TP + FP + FN)  
  
    print(f"\nClass {i}")  
    print("TP:", TP)  
    print("FP:", FP)  
    print("FN:", FN)  
    print("TN:", TN)
```

Class 0

TP: 16

FP: 0

FN: 0

TN: 29

Class 1

TP: 14

FP: 5

FN: 4

TN: 22

Class 2

TP: 6

FP: 4

FN: 5

TN: 30

```
In [118... accuracy = accuracy_score(y_test, y_pred)  
error_rate = 1 - accuracy  
  
print("\nAccuracy:", accuracy)  
print("Error Rate:", error_rate)
```

Accuracy: 0.8

Error Rate: 0.19999999999999996

```
In [120... print("\nClassification Report:\n")  
print(classification_report(y_test, y_pred))
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	16
1	0.74	0.78	0.76	18
2	0.60	0.55	0.57	11
accuracy			0.80	45
macro avg	0.78	0.77	0.78	45
weighted avg	0.80	0.80	0.80	45

In [122... *# Decision Boundary with Legend*

```

X_set, y_set = X_train, y_train

X1, X2 = np.meshgrid(
    np.arange(X_set[:, 0].min()-1, X_set[:, 0].max()+1, 0.01),
    np.arange(X_set[:, 1].min()-1, X_set[:, 1].max()+1, 0.01)
)

plt.figure(figsize=(7,6))

# Background prediction regions
plt.contourf(
    X1, X2,
    model.predict(np.array([X1.ravel(), X2.ravel()]).T).reshape(X1.shape),
    alpha=0.7,
    cmap=ListedColormap(('red', 'green', 'black'))
)

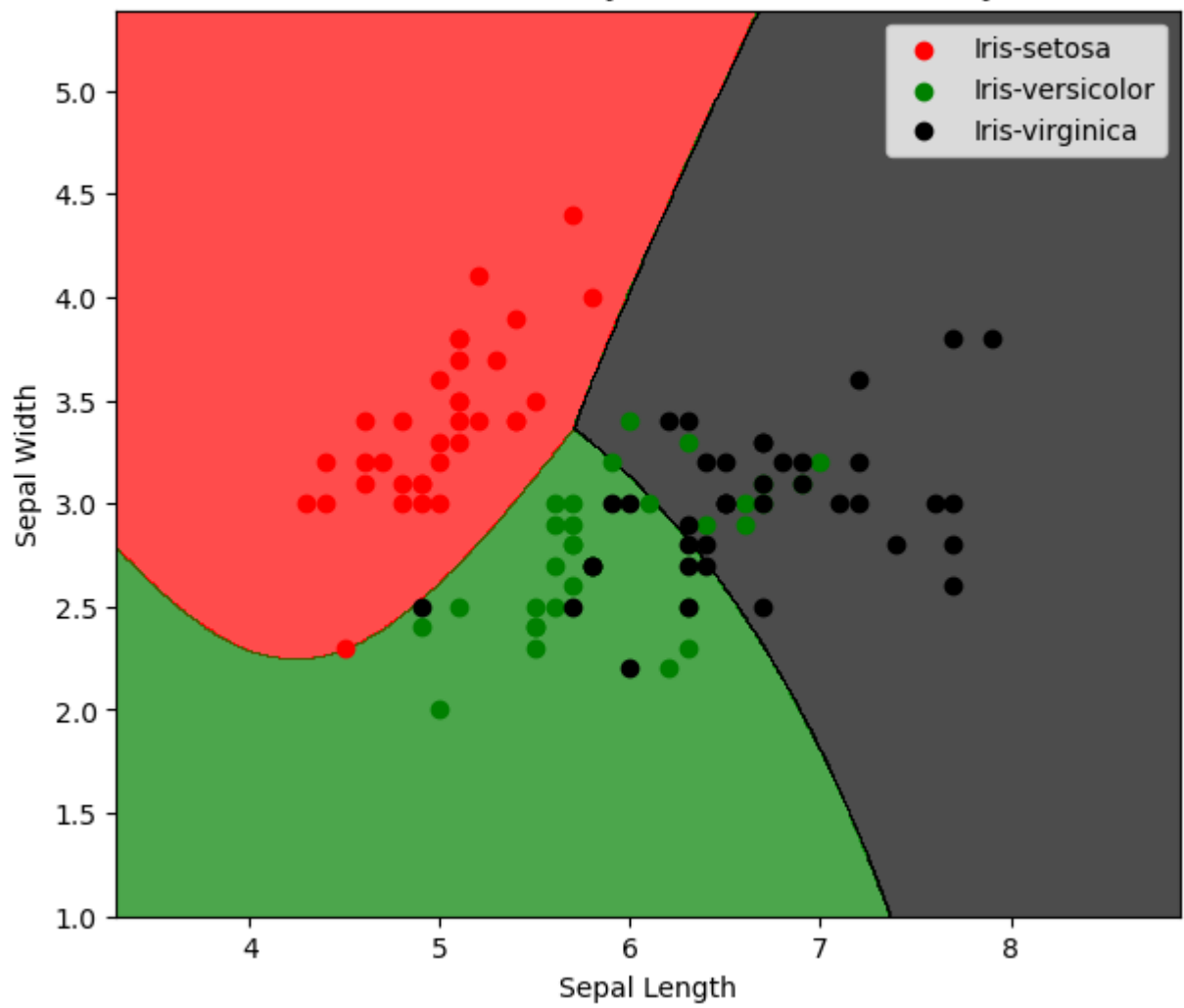
# Plot each class separately for proper legend
colors = ('red', 'green', 'black')
labels = le.classes_

for i, color in enumerate(colors):
    plt.scatter(
        X_set[y_set == i, 0],
        X_set[y_set == i, 1],
        c=color,
        label=labels[i]
    )

plt.title("Gaussian Naïve Bayes - Decision Boundary")
plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.legend()
plt.show()

```

Gaussian Naïve Bayes - Decision Boundary



In []:

```
In [95]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
```

```
In [97]: df = pd.read_csv("emails.csv")

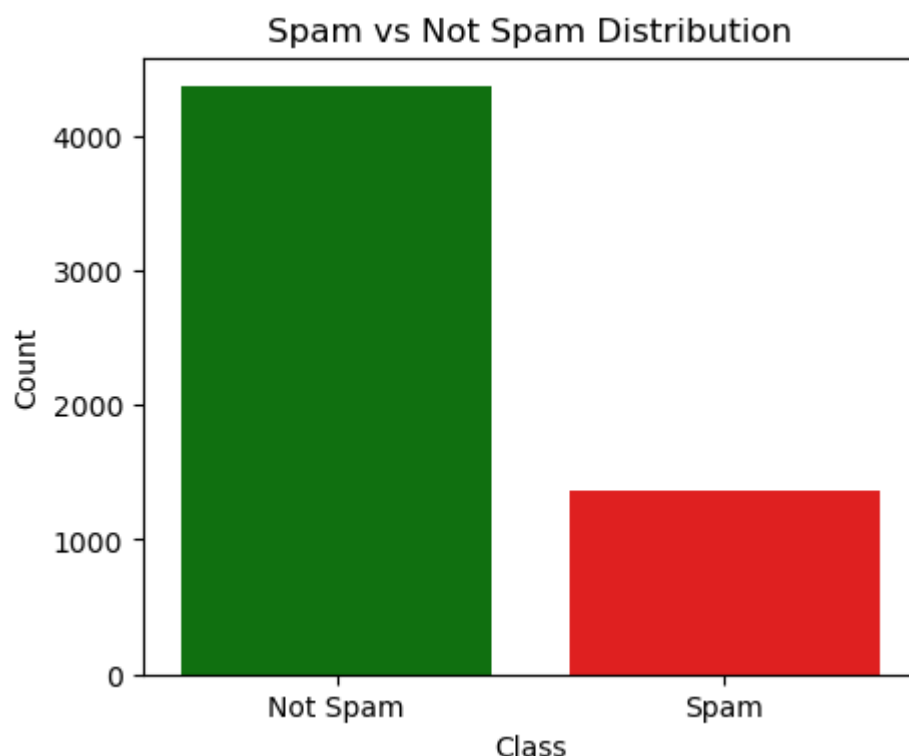
# Rename column if needed
if 'spam_or_not' in df.columns:
    df.rename(columns={'spam_or_not': 'spam'}, inplace=True)

df.head()
```

```
Out[97]:
```

	text	spam
0	Subject: naturally irresistible your corporate...	1
1	Subject: the stock trading gunslinger fanny i...	1
2	Subject: unbelievable new homes made easy im ...	1
3	Subject: 4 color printing special request add...	1
4	Subject: do not have money , get software cds ...	1

```
In [99]: plt.figure(figsize=(5,4))
sns.countplot(x='spam', data=df, hue='spam', palette=['green', 'red'], legend=False)
plt.xticks([0,1], ['Not Spam', 'Spam'])
plt.title("Spam vs Not Spam Distribution")
plt.xlabel("Class")
plt.ylabel("Count")
plt.show()
```



```
In [101... X = df['text']
y = df['spam']
```

```
In [103... X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=0
)
```

```
In [105... vectorizer = TfidfVectorizer(stop_words='english')

X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)
```

```
In [106... model = MultinomialNB()
model.fit(X_train_vec, y_train)

y_pred = model.predict(X_test_vec)
```

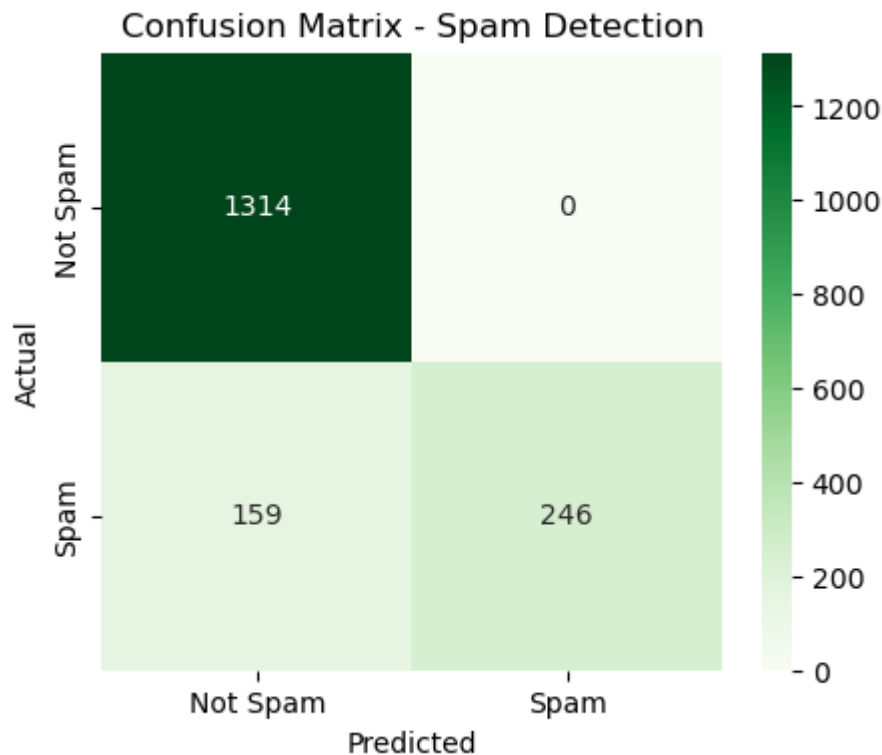
```
In [109... cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
```

Confusion Matrix:

```
[[1314   0]
 [ 159 246]]
```

```
In [111... plt.figure(figsize=(5,4))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Greens',
    xticklabels=['Not Spam', 'Spam'],
    yticklabels=['Not Spam', 'Spam']
)

plt.title("Confusion Matrix - Spam Detection")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



```
In [113... TN, FP, FN, TP = cm.ravel()

print("True Positive (TP) - Spam correctly predicted:", TP)
print("False Positive (FP) - Not Spam predicted as Spam:", FP)
```

```
print("True Negative (TN) - Not Spam correctly predicted:", TN)
print("False Negative (FN) - Spam predicted as Not Spam:", FN)
```

True Positive (TP) - Spam correctly predicted: 246
False Positive (FP) - Not Spam predicted as Spam: 0
True Negative (TN) - Not Spam correctly predicted: 1314
False Negative (FN) - Spam predicted as Not Spam: 159

```
In [115... accuracy = accuracy_score(y_test, y_pred)
error_rate = 1 - accuracy

print("\nAccuracy:", accuracy)
print("Error Rate:", error_rate)
```

Accuracy: 0.9075043630017452
Error Rate: 0.0924956369982548

```
In [117... print("\nClassification Report:\n")
print(classification_report(
    y_test,
    y_pred,
    target_names=['Not Spam', 'Spam']
))
```

Classification Report:

	precision	recall	f1-score	support
Not Spam	0.89	1.00	0.94	1314
Spam	1.00	0.61	0.76	405
accuracy			0.91	1719
macro avg	0.95	0.80	0.85	1719
weighted avg	0.92	0.91	0.90	1719

In []:


```
In [1]: import nltk
nltk.download('punkt')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')
nltk.download('stopwords')
from nltk import sent_tokenize
from nltk import word_tokenize
from nltk.corpus import stopwords
```

```
[nltk_data] Downloading package punkt to
[nltk_data] C:\Users\RBS\AppData\Roaming\nltk_data...
[nltk_data] Unzipping tokenizers\punkt.zip.
[nltk_data] Downloading package wordnet to
[nltk_data] C:\Users\RBS\AppData\Roaming\nltk_data...
[nltk_data] Unzipping corpora\wordnet.zip.
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] C:\Users\RBS\AppData\Roaming\nltk_data...
[nltk_data] Unzipping taggers\averaged_perceptron_tagger.zip.
[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\RBS\AppData\Roaming\nltk_data...
[nltk_data] Unzipping corpora\stopwords.zip.
```

```
In [2]: text='Real madrid is set to win the UCL for the season . Benzema might win Balon dor
```

```
In [3]: tokens_sents = nltk.sent_tokenize(text)
print(tokens_sents)
```

```
['Real madrid is set to win the UCL for the season .', 'Benzema might win Balon dor .', 'Salah might be the runner up']
```

```
In [4]: tokens_words = nltk.word_tokenize(text)
print(tokens_words)
```

```
['Real', 'madrid', 'is', 'set', 'to', 'win', 'the', 'UCL', 'for', 'the', 'season', '.', 'Benzema', 'might', 'win', 'Balon', 'dor', '.', 'Salah', 'might', 'be', 'the', 'runner', 'up']
```

```
In [5]: from nltk.stem import PorterStemmer
from nltk.stem.snowball import SnowballStemmer
from nltk.stem import LancasterStemmer
```

```
In [9]: stem=[]
for i in tokens_words:
    ps = PorterStemmer()
    stem_word= ps.stem(i)
    stem.append(stem_word)
print(stem)
```

```
['real', 'madrid', 'is', 'set', 'to', 'win', 'the', 'ucl', 'for', 'the', 'season', '.', 'benzema', 'might', 'win', 'balon', 'dor', '.', 'salah', 'might', 'be', 'the', 'runner', 'up']
```

Lemmatization

```
In [10]: import nltk
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
```

```
In [11]: lemmatized_output = ' '.join([lemmatizer.lemmatize(w) for w in stem])
print(lemmatized_output)
```

```
real madrid is set to win the ucl for the season . benzema might win balon dor . salah mightbe the runner up
```

```
In [12]: leme=[]
         for i in stem:
             lemetized_word=lemmatizer.lemmatize(i)
             leme.append(lemetized_word)
         print(leme)

['real', 'madrid', 'is', 'set', 'to', 'win', 'the', 'ucl', 'for', 'the', 'season',
 '.', 'benzema', 'might', 'win', 'balon', 'dor', '.', 'salah', 'might', 'be', 'the', 'runner', 'up']
```

Part of Speech Tagging

```
In [13]: print("Parts of Speech: ", nltk.pos_tag(leme))

Parts of Speech: [('real', 'JJ'), ('madrid', 'NN'), ('is', 'VBZ'), ('set', 'VBN'),
 ('to', 'TO'), ('win', 'VB'), ('the', 'DT'), ('ucl', 'NN'), ('for', 'IN'), ('the', 'DT'),
 ('season', 'NN'), ('.', '.'), ('benzema', 'NN'), ('might', 'MD'), ('win', 'VB'),
 ('balon', 'NN'), ('dor', 'NN'), ('.', '.'), ('salah', 'NN'), ('might', 'MD'), ('be', 'VB'),
 ('the', 'DT'), ('runner', 'NN'), ('up', 'RP')]
```

Stop Word

```
In [14]: sw_nltk = stopwords.words('english')
         print(sw_nltk)

['i', 'me', 'my', 'myself', 'we', 'our', 'ours', 'ourselves', 'you', "you're", "you've",
 "you'll", "you'd", 'your', 'yours', 'yourself', 'yourselves', 'he', 'him', 'his',
 'himself', 'she', "she's", 'her', 'hers', 'herself', 'it', "it's", 'its', 'itself', 'they',
 'them', 'their', 'theirs', 'themselves', 'what', 'which', 'who', 'whom', 'this', 'that',
 "that'll", 'these', 'those', 'am', 'is', 'are', 'was', 'were', 'be', 'been', 'being',
 'have', 'has', 'had', 'having', 'do', 'does', 'did', 'doing', 'a', 'an', 'the', 'and',
 'but', 'if', 'or', 'because', 'as', 'until', 'while', 'of', 'at', 'by', 'for', 'with',
 'about', 'against', 'between', 'into', 'through', 'during', 'before', 'after', 'above',
 'below', 'to', 'from', 'up', 'down', 'in', 'out', 'on', 'off', 'over', 'under', 'again',
 'further', 'then', 'once', 'here', 'there', 'when', 'where', 'why', 'how', 'all', 'any',
 'both', 'each', 'few', 'more', 'most', 'other', 'some', 'such', 'no', 'nor', 'not', 'only',
 'own', 'same', 'so', 'than', 'too', 'very', 's', 't', 'can', 'will', 'just', 'don', "don't",
 'should', "should've", 'now', 'd', 'll', 'm', 'o', 're', 've', 'y', 'ain', 'aren', "aren't",
 'couldn', "couldn't", 'didn', "didn't", 'doesn', "doesn't", 'hadn', "hadn't", 'hasn',
 "hasn't", 'haven', "haven't", 'isn', "isn't", 'ma', 'mightn', "mightn't", 'mustn',
 "mustn't", 'needn', "needn't", 'shan', "shan't", 'shouldn', "shouldn't", 'wasn',
 "wasn't", 'weren', "weren't", 'won', "won't", 'wouldn', "wouldn't"]
```

```
In [15]: words = [word for word in text.split() if word.lower() not in sw_nltk]
         new_text = " ".join(words)
         print(new_text)

Real madrid set win UCL season . Benzema might win Balon dor . Salah might runner
```

```
In [34]: # Import libraries
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Load dataset
titanic = sns.load_dataset('titanic')

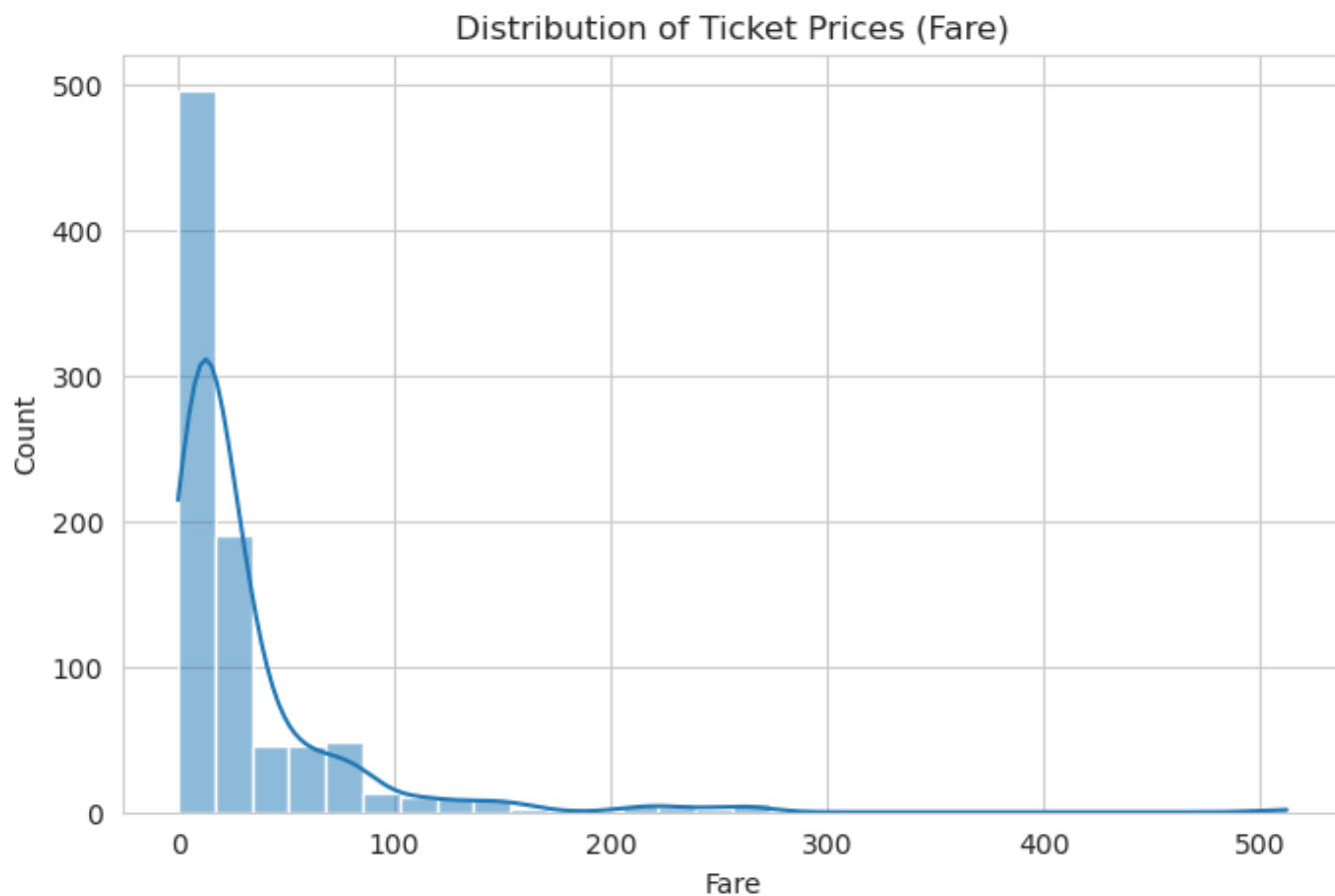
# Set style
sns.set_style("whitegrid")

# Preview data
titanic.head()
```

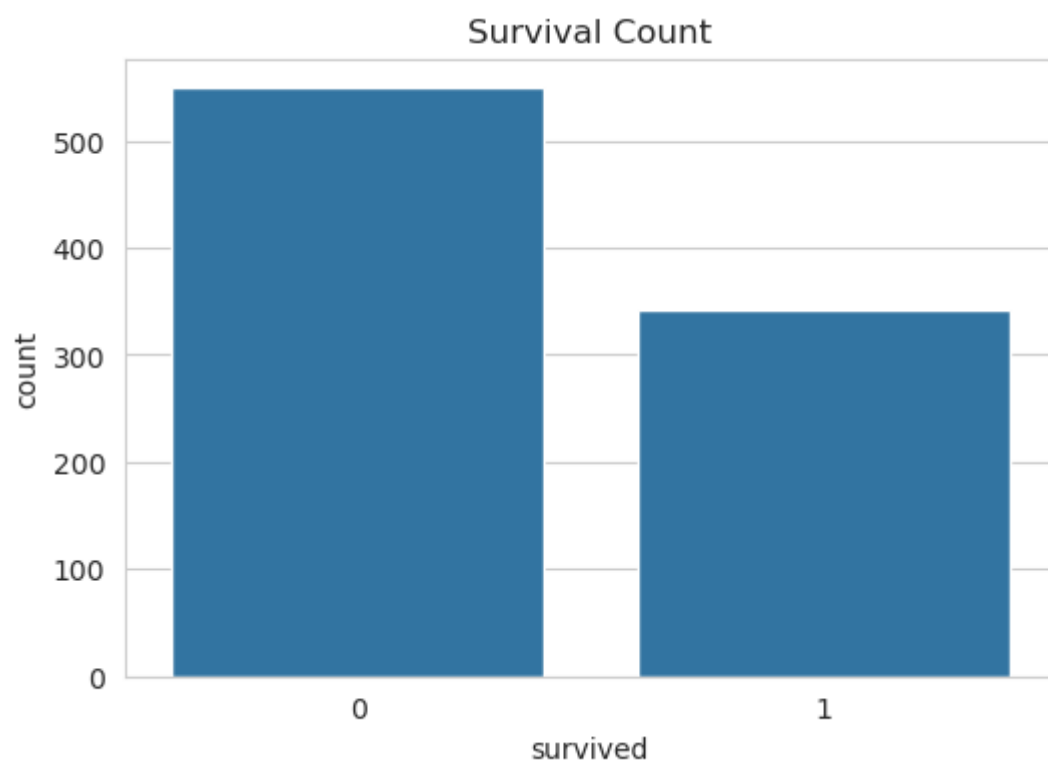
```
Out[34]:
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult
0	0	3	male	22.0	1	0	7.2500	S	Third	man	
1	1	1	female	38.0	1	0	71.2833	C	First	woman	
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	
3	1	1	female	35.0	1	0	53.1000	S	First	woman	
4	0	3	male	35.0	0	0	8.0500	S	Third	man	

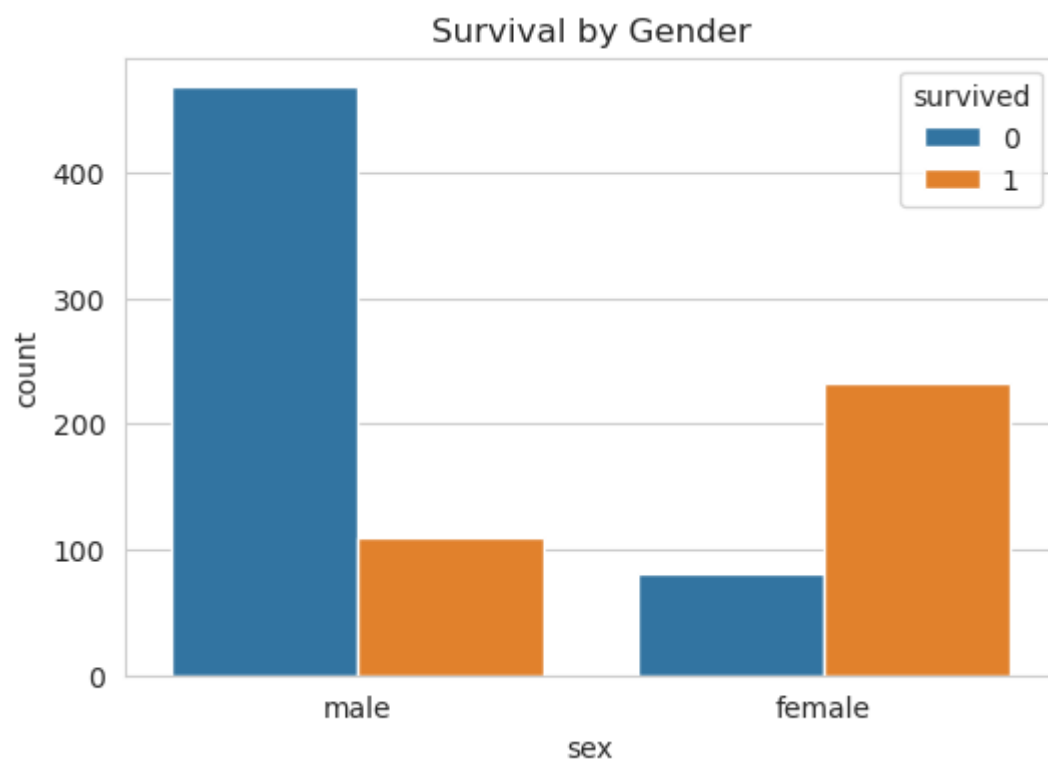
```
In [36]: plt.figure(figsize=(8,5))
sns.histplot(data=titanic, x='fare', bins=30, kde=True)
plt.title('Distribution of Ticket Prices (Fare)')
plt.xlabel('Fare')
plt.ylabel('Count')
plt.show()
```



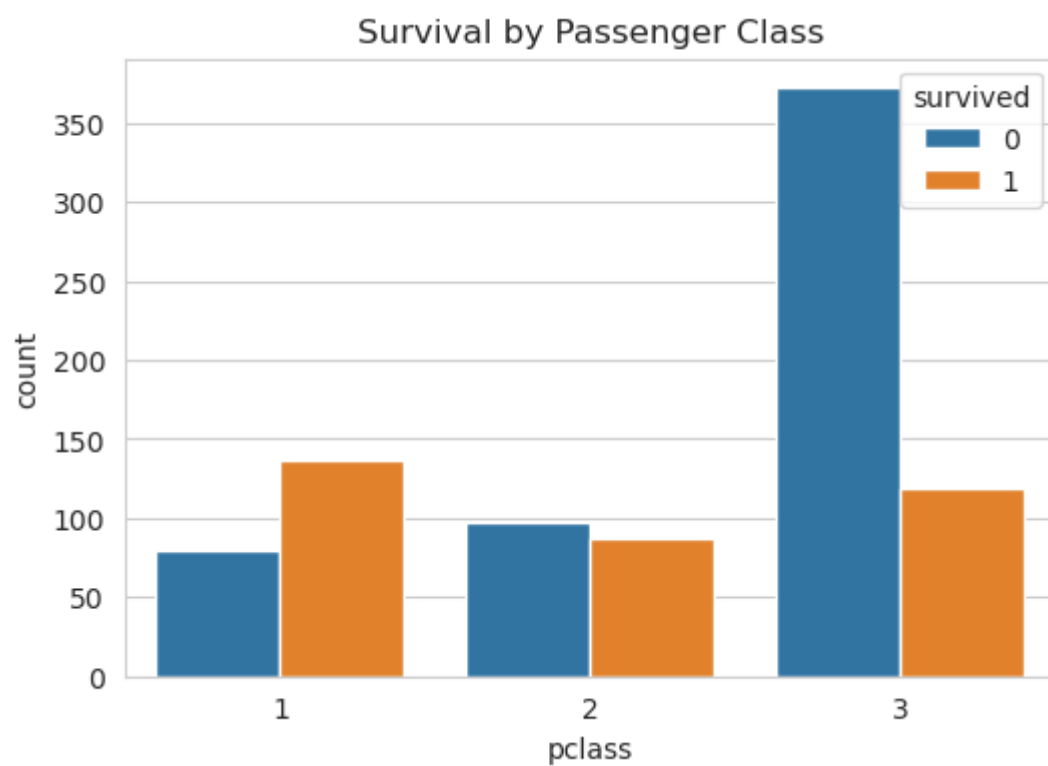
```
In [38]: plt.figure(figsize=(6,4))
sns.countplot(data=titanic, x='survived')
plt.title('Survival Count')
plt.show()
```



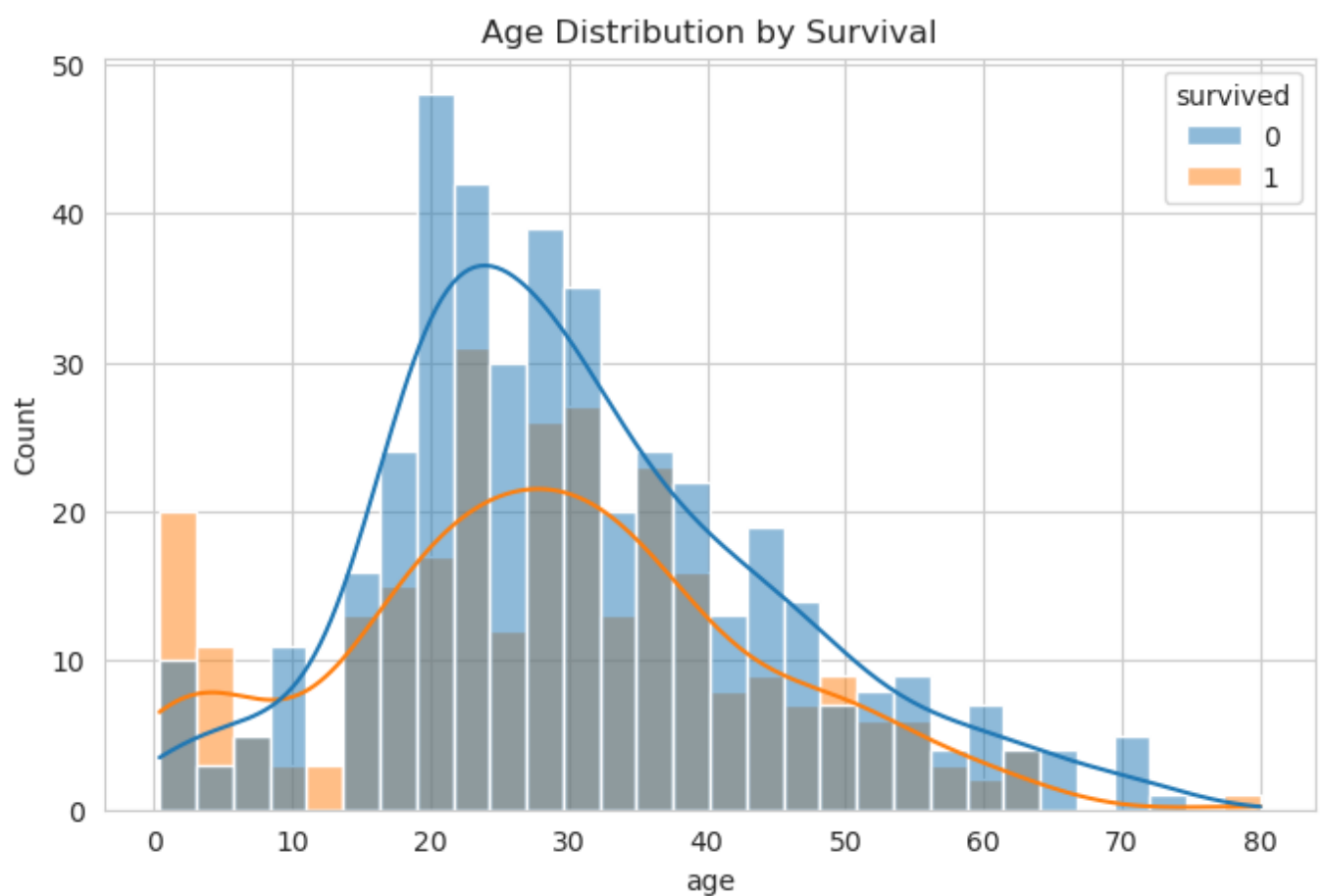
```
In [40]: plt.figure(figsize=(6,4))
sns.countplot(data=titanic, x='sex', hue='survived')
plt.title('Survival by Gender')
plt.show()
```



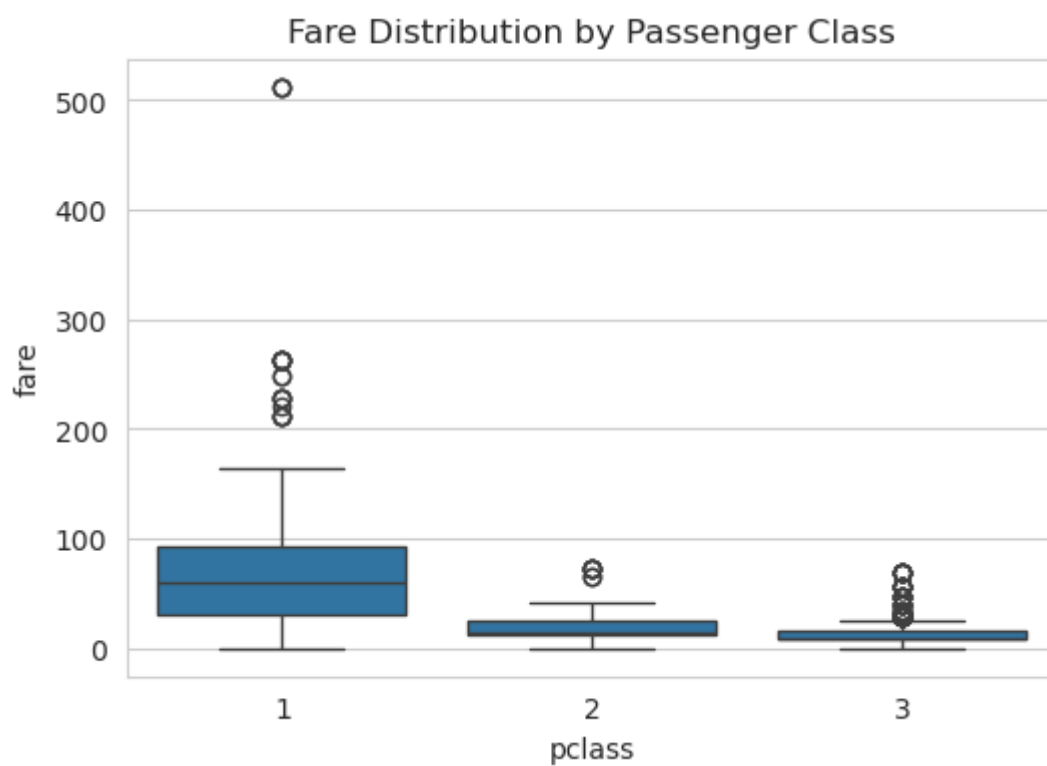
```
In [41]: plt.figure(figsize=(6,4))
sns.countplot(data=titanic, x='pclass', hue='survived')
plt.title('Survival by Passenger Class')
plt.show()
```



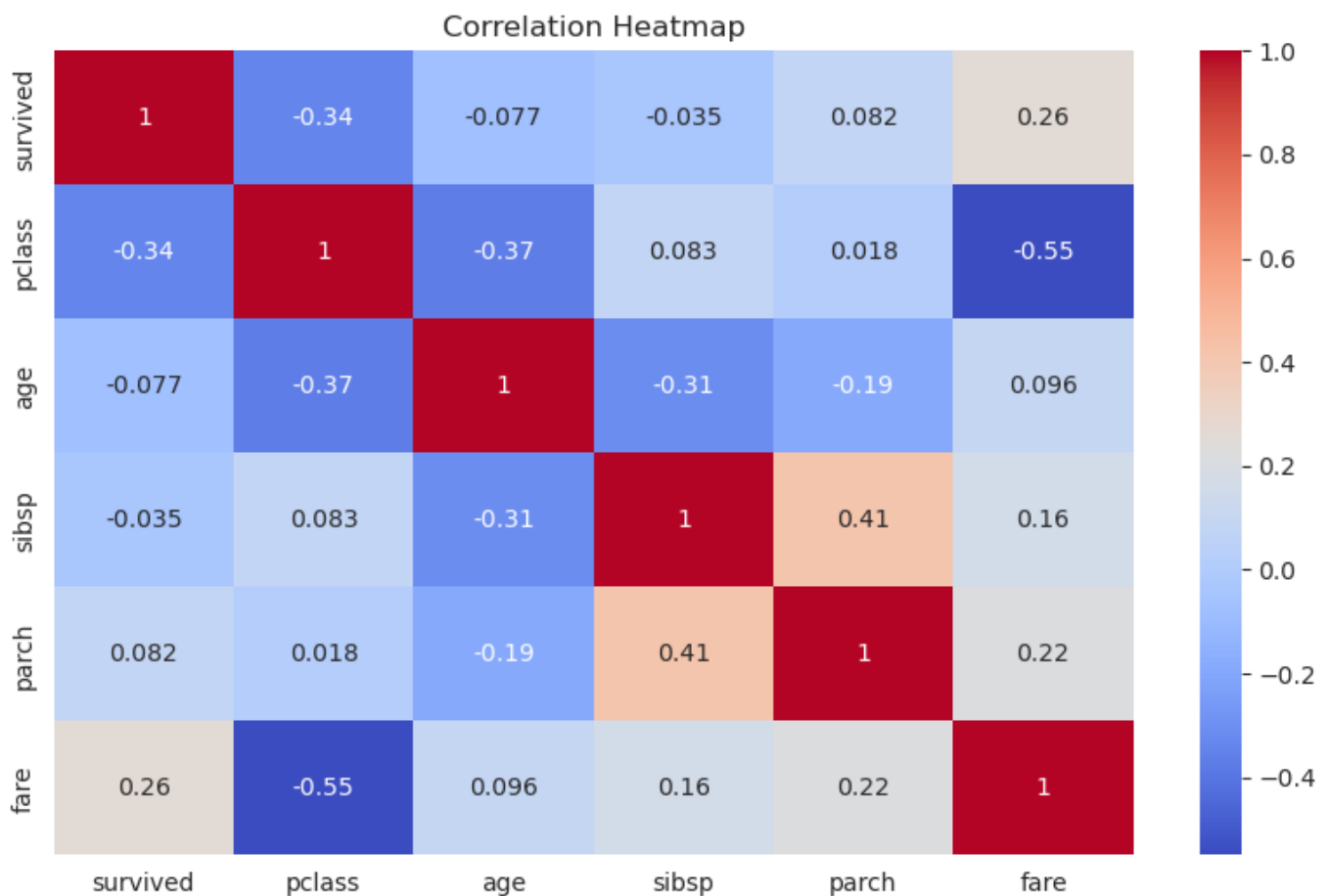
```
In [43]: plt.figure(figsize=(8,5))
sns.histplot(data=titanic, x='age', hue='survived', bins=30, kde=True)
plt.title('Age Distribution by Survival')
plt.show()
```



```
In [45]: plt.figure(figsize=(6,4))
sns.boxplot(data=titanic, x='pclass', y='fare')
plt.title('Fare Distribution by Passenger Class')
plt.show()
```

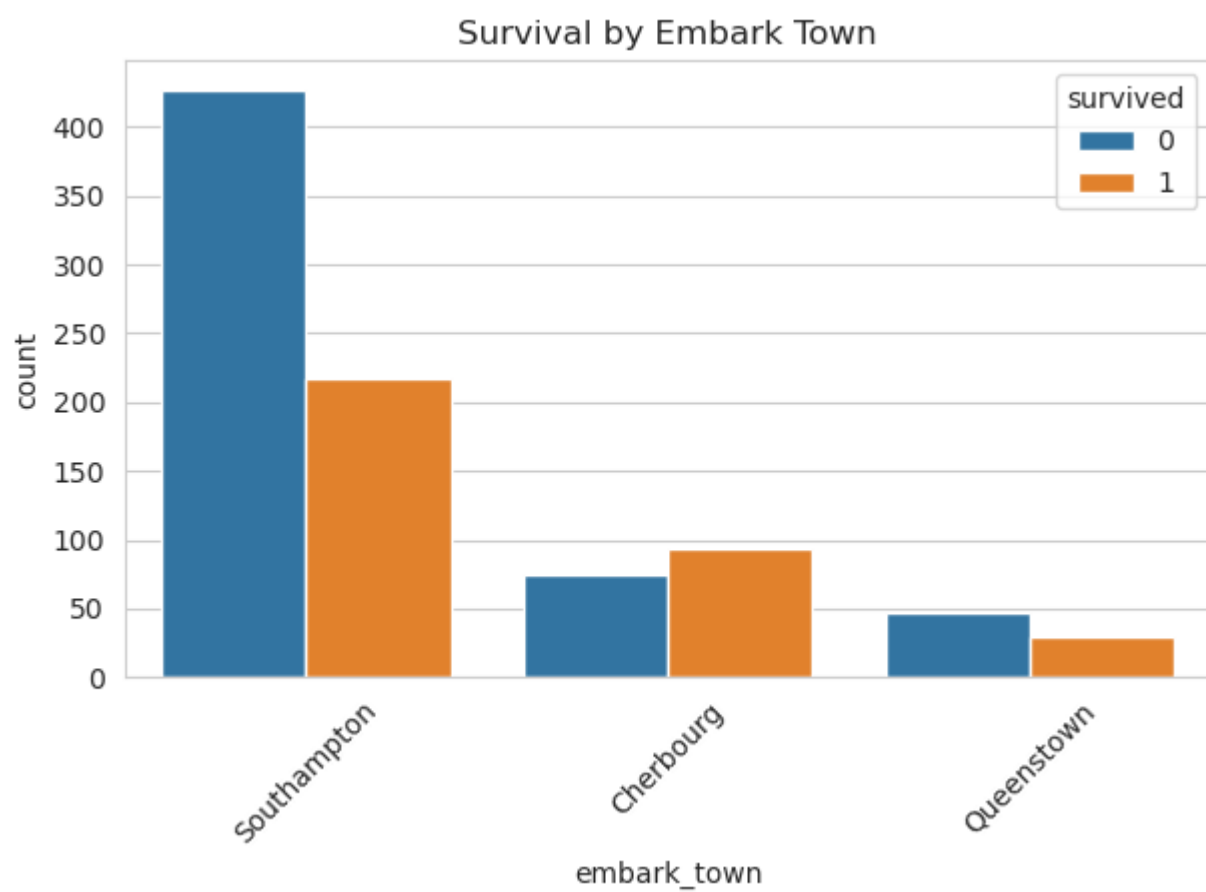


```
In [47]: plt.figure(figsize=(10,6))
numeric_data = titanic.select_dtypes(include=['float64', 'int64'])
sns.heatmap(numeric_data.corr(), annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```

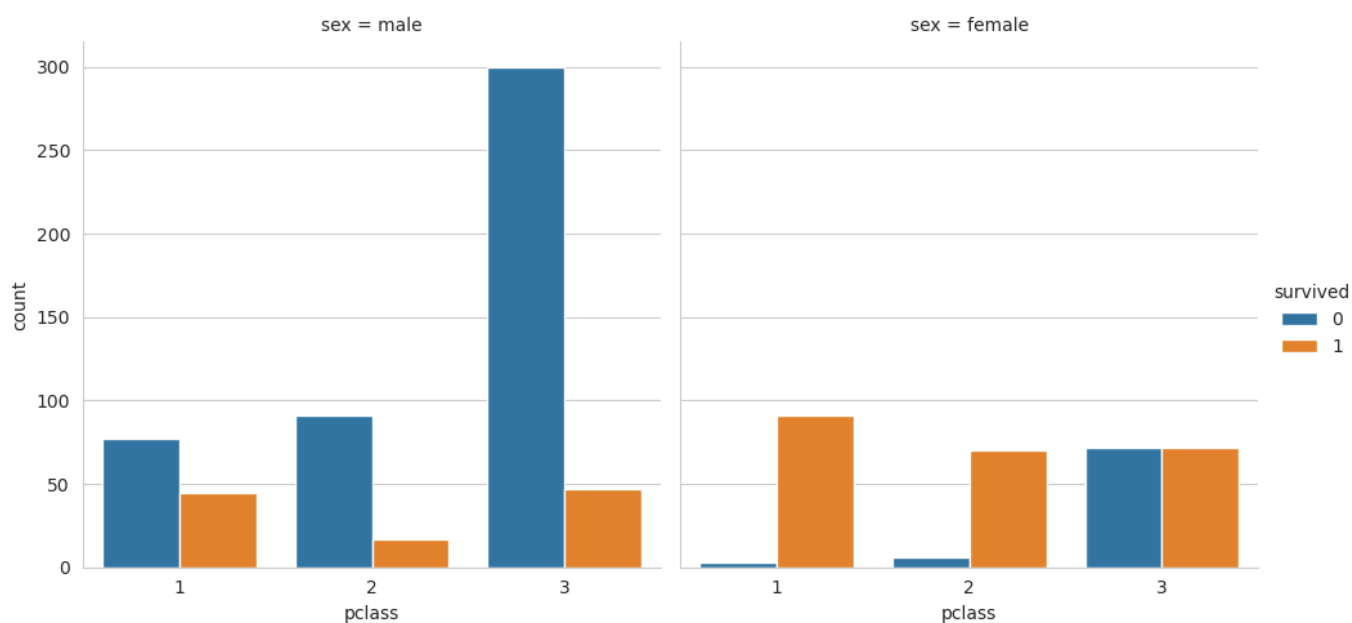


```
In [ ]:
```

```
In [50]: plt.figure(figsize=(7,4))
sns.countplot(data=titanic, x='embark_town', hue='survived')
plt.xticks(rotation=45)
plt.title('Survival by Embark Town')
plt.show()
```



```
In [52]: sns.catplot(data=titanic, x='pclass', hue='survived', col='sex', kind='count')  
plt.show()
```



```
In [ ]:
```

```
In [42]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

data = sns.load_dataset('titanic')

data.head()
```

Out[42]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult
0	0	3	male	22.0	1	0	7.2500	S	Third	man	
1	1	1	female	38.0	1	0	71.2833	C	First	woman	
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	
3	1	1	female	35.0	1	0	53.1000	S	First	woman	
4	0	3	male	35.0	0	0	8.0500	S	Third	man	

```
In [44]: data.describe()
```

Out[44]:

	survived	pclass	age	sibsp	parch	fare
count	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

```
In [46]: data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column          Non-Null Count  Dtype
---  -
0   survived        891 non-null   int64
1   pclass          891 non-null   int64
2   sex             891 non-null   object
3   age            714 non-null   float64
4   sibsp          891 non-null   int64
5   parch          891 non-null   int64
6   fare           891 non-null   float64
7   embarked       889 non-null   object
8   class          891 non-null   category
9   who            891 non-null   object
10  adult_male     891 non-null   bool
11  deck          203 non-null   category
12  embark_town    889 non-null   object
13  alive         891 non-null   object
14  alone         891 non-null   bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB
```

```
In [48]: data.isnull().sum()
```



```
Out[48]: survived      0
         pclass        0
         sex           0
         age          177
         sibsp         0
         parch         0
         fare          0
         embarked      2
         class         0
         who           0
         adult_male     0
         deck          688
         embark_town     2
         alive         0
         alone         0
         dtype: int64
```

```
In [54]: data['age'] = data['age'].fillna(np.mean(data['age']))
```

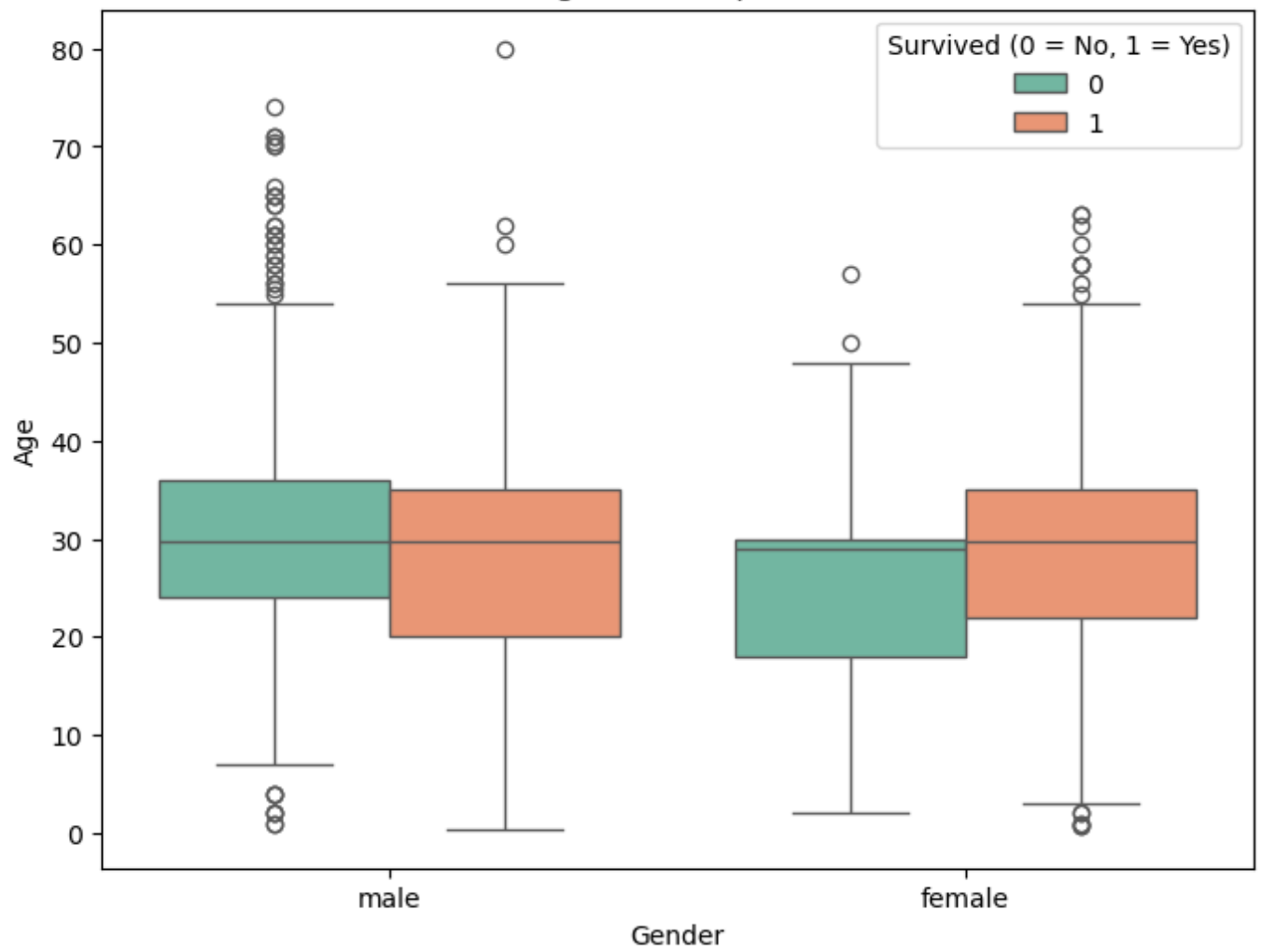
```
In [56]: plt.figure(figsize=(8,6))

sns.boxplot(
    x='sex',
    y='age',
    hue='survived',
    data=data,
    palette='Set2'
)

plt.title('Plot for Distribution of Age with Respect to Gender and Survival')
plt.xlabel('Gender')
plt.ylabel('Age')
plt.legend(title='Survived (0 = No, 1 = Yes)')

plt.show()
```

Plot for Distribution of Age with Respect to Gender and Survival



In []:

```
In [19]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [20]: data = pd.read_csv('https://gist.githubusercontent.com/curran/a08a1080b88344b0c8a7/ra
data
```

```
Out[20]:
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

150 rows × 5 columns

```
In [21]: data.head()
```

```
Out[21]:
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

```
In [22]: data.describe()
```

```
Out[22]:
```

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

```
In [23]: data.describe(include = 'object')
```

Out[23]:

species	
count	150
unique	3
top	setosa
freq	50

```
In [24]: data.isnull().sum()
```

```
Out[24]: sepal_length    0
sepal_width    0
petal_length    0
petal_width    0
species        0
dtype: int64
```

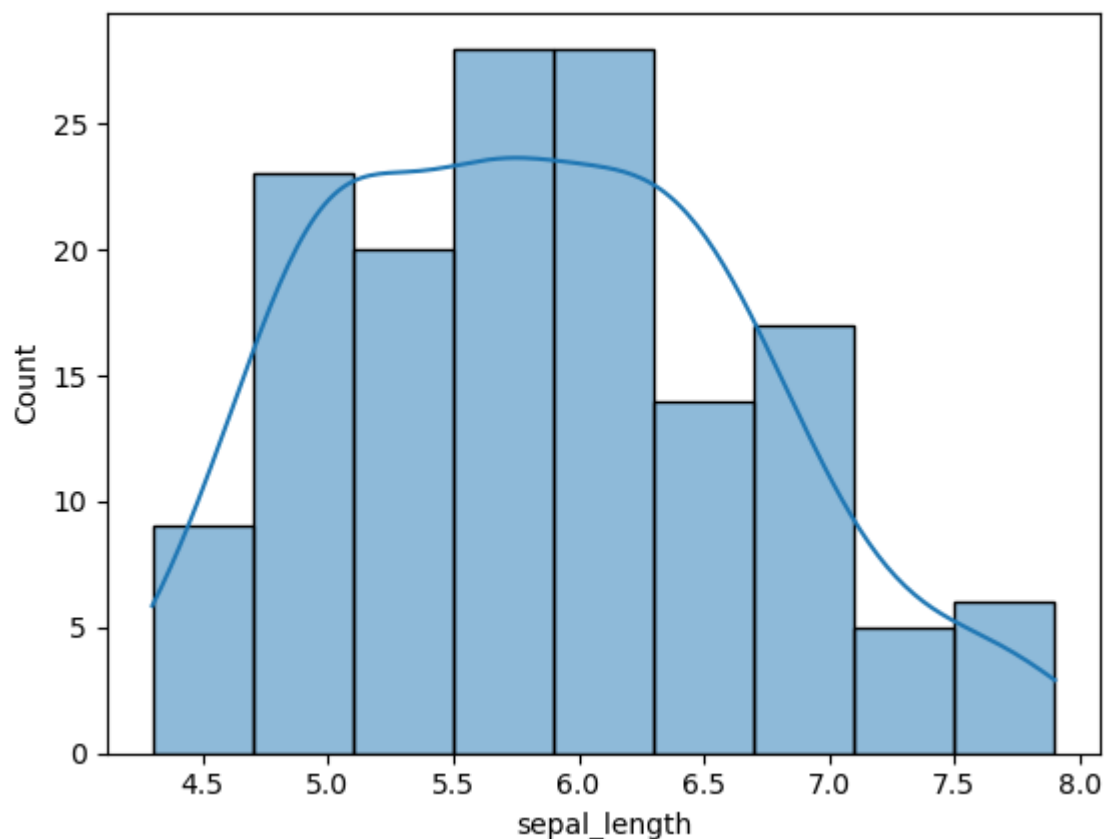
```
In [25]: print("\n\nThe features in the dataset are as follows : ")
print("1. Sepal length : ", data['sepal_length'].dtype)
print("2. Sepal width : ", data['sepal_width'].dtype)
print("3. Petal length : ", data['petal_length'].dtype)
print("4. Petal width : ", data['petal_width'].dtype)
print("5. Species : ", data['species'].dtype)
```

The features in the dataset are as follows :

1. Sepal length : float64
2. Sepal width : float64
3. Petal length : float64
4. Petal width : float64
5. Species : object

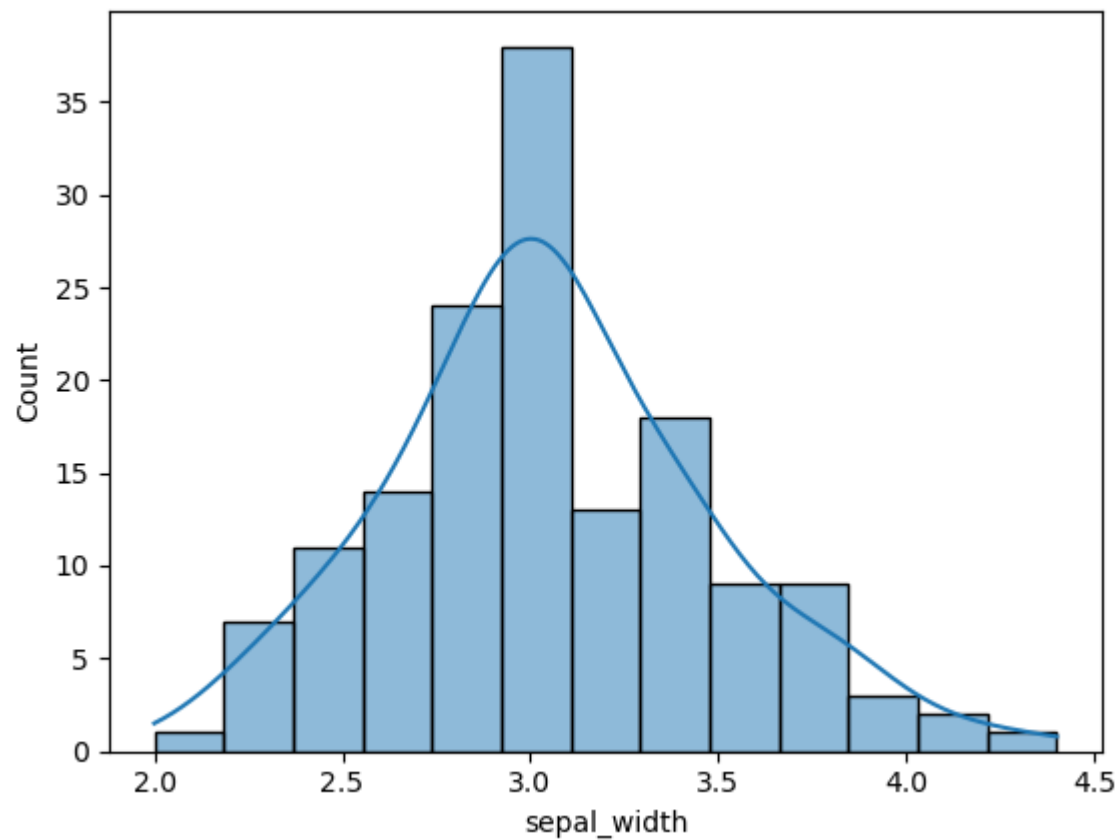
```
In [26]: sns.histplot(x = data['sepal_length'], kde=True)
```

```
Out[26]: <Axes: xlabel='sepal_length', ylabel='Count'>
```



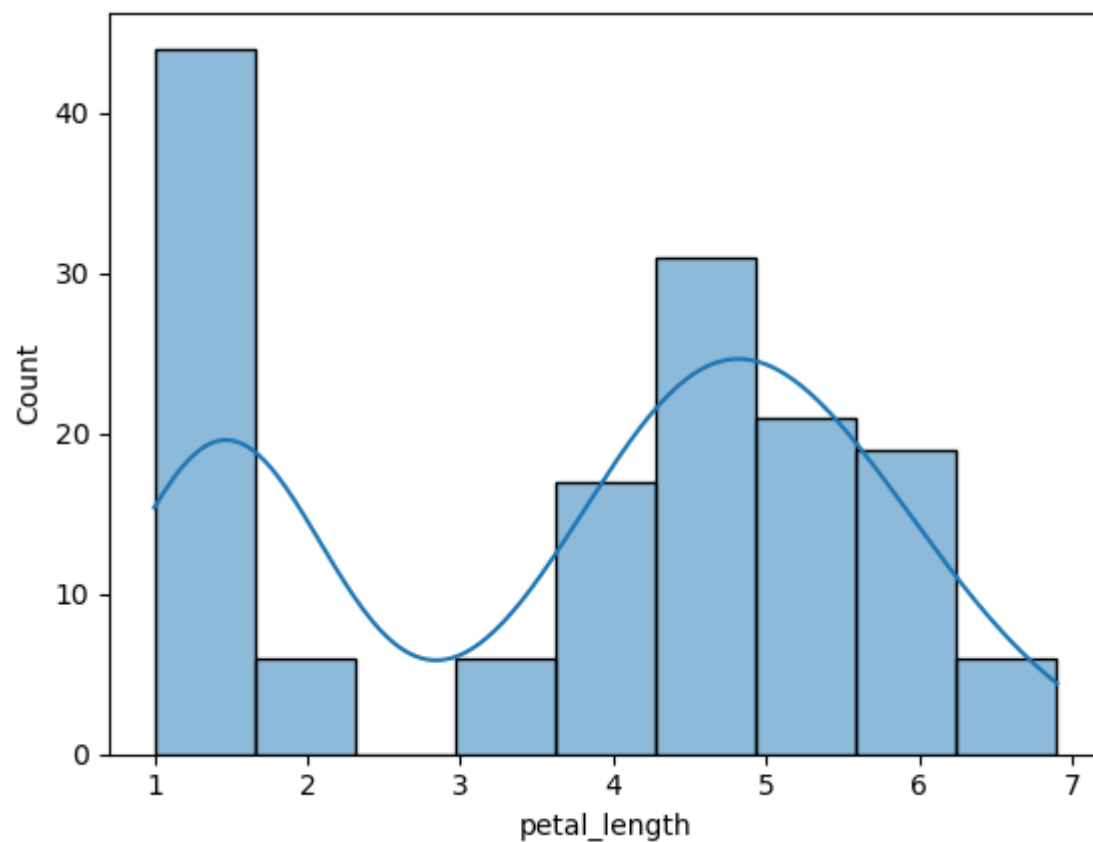
```
In [27]: sns.histplot(x = data['sepal_width'], kde=True)
```

Out[27]: <Axes: xlabel='sepal_width', ylabel='Count'>



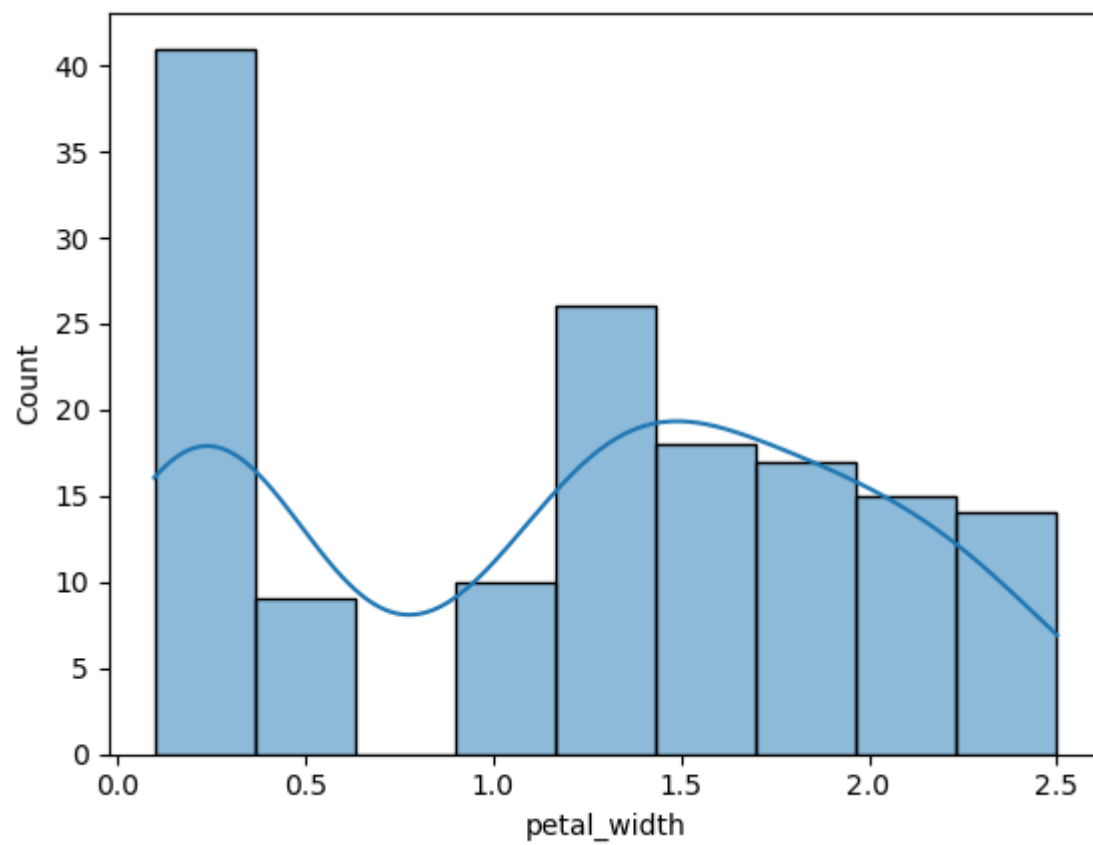
```
In [28]: sns.histplot(x = data['petal_length'], kde=True)
```

Out[28]: <Axes: xlabel='petal_length', ylabel='Count'>



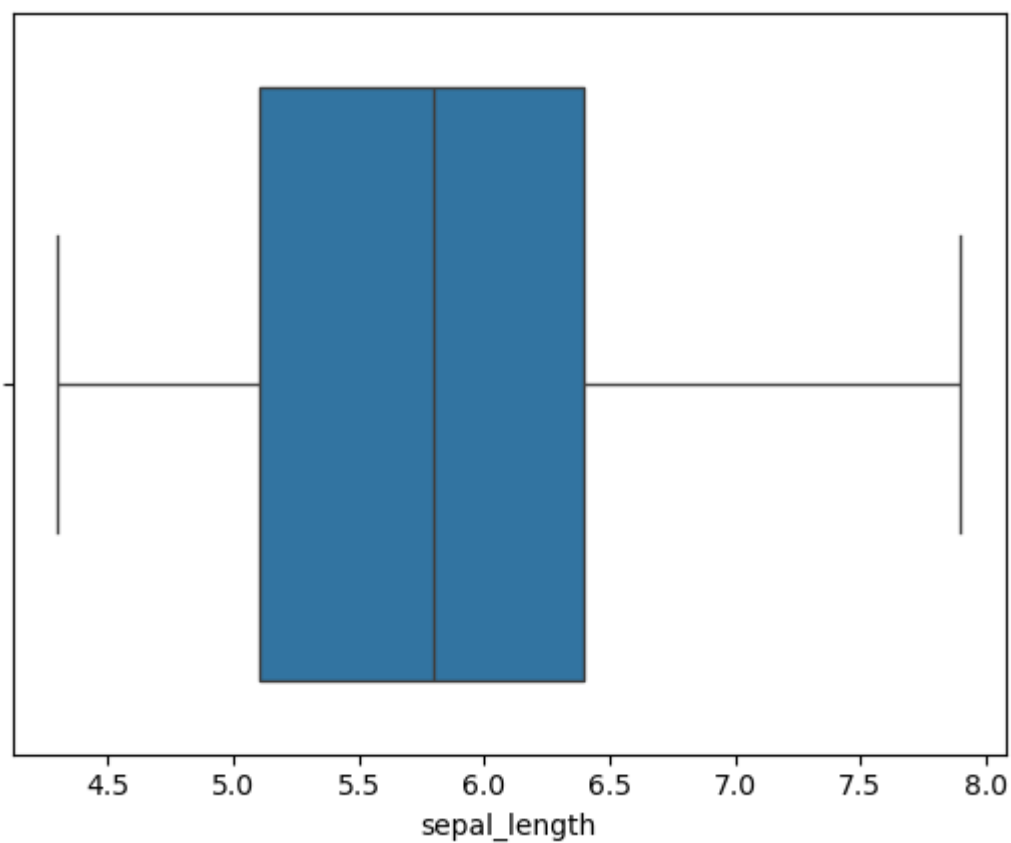
```
In [29]: sns.histplot(x = data['petal_width'], kde=True)
```

Out[29]: <Axes: xlabel='petal_width', ylabel='Count'>



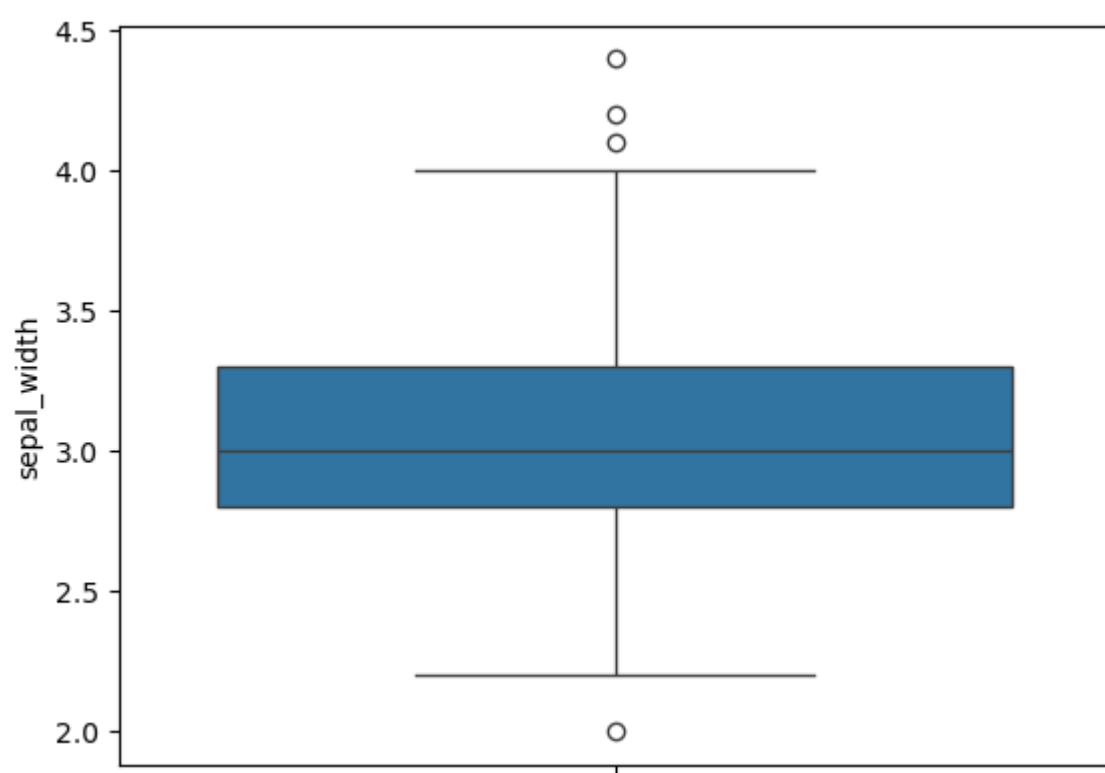
```
In [30]: sns.boxplot(x=data['sepal_length'])
```

```
Out[30]: <Axes: xlabel='sepal_length'>
```



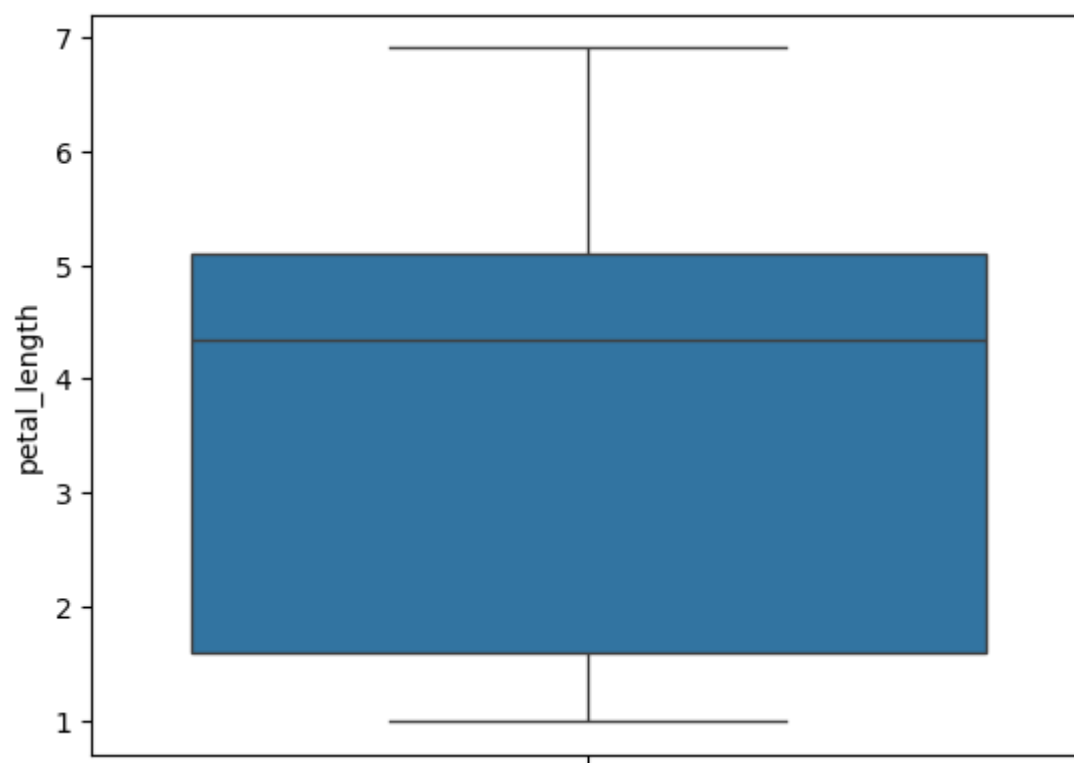
```
In [31]: sns.boxplot(data['sepal_width'])
```

```
Out[31]: <Axes: ylabel='sepal_width'>
```



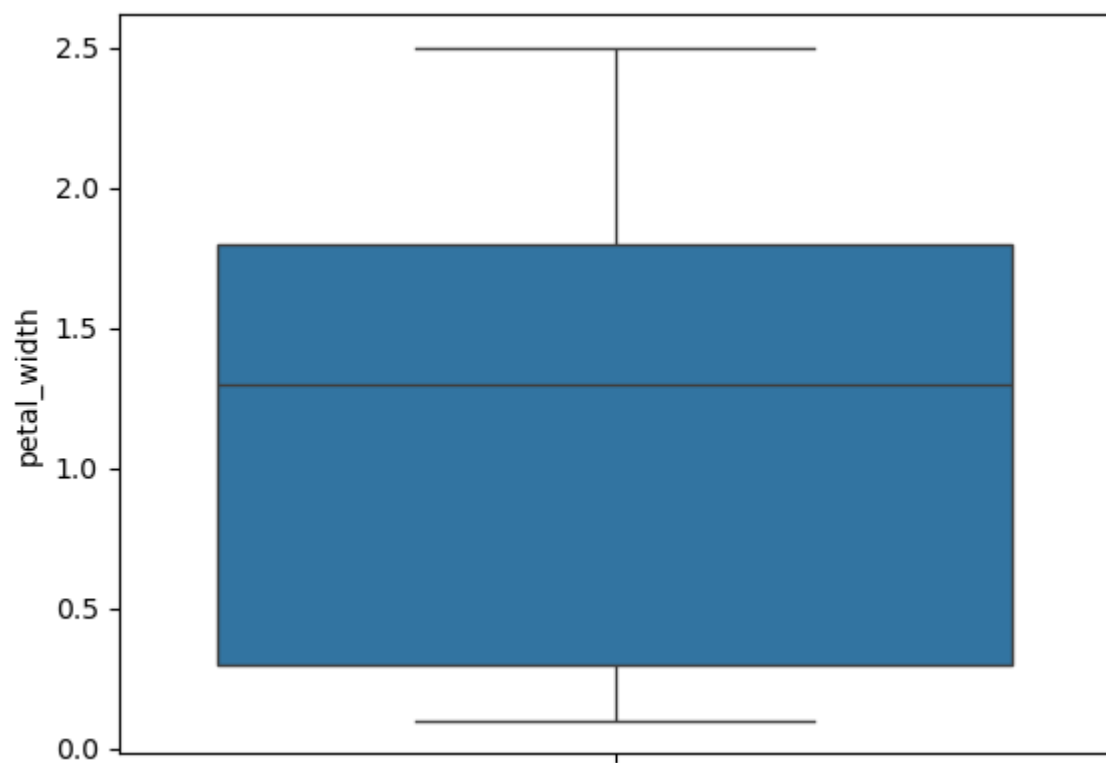
```
In [32]: sns.boxplot(data['petal_length'])
```

```
Out[32]: <Axes: ylabel='petal_length'>
```



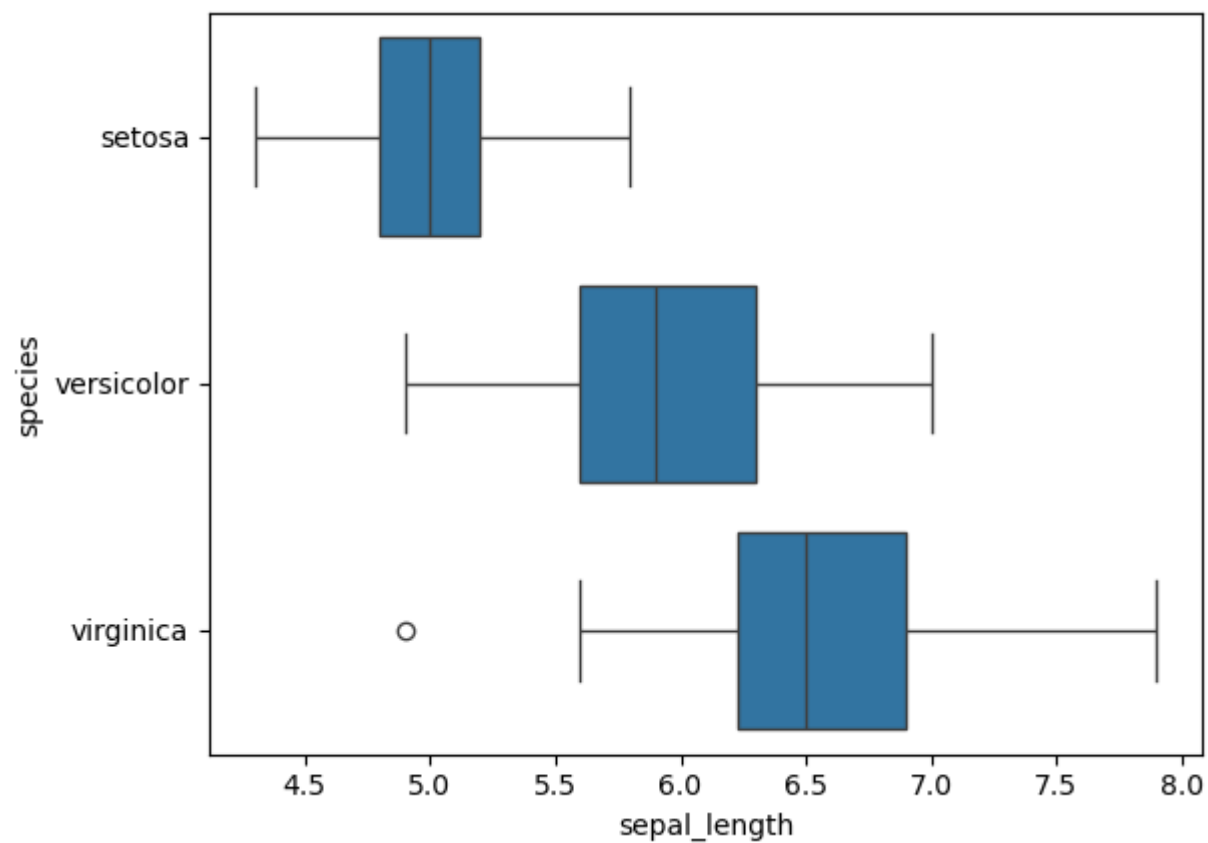
```
In [33]: sns.boxplot(data['petal_width'])
```

```
Out[33]: <Axes: ylabel='petal_width'>
```



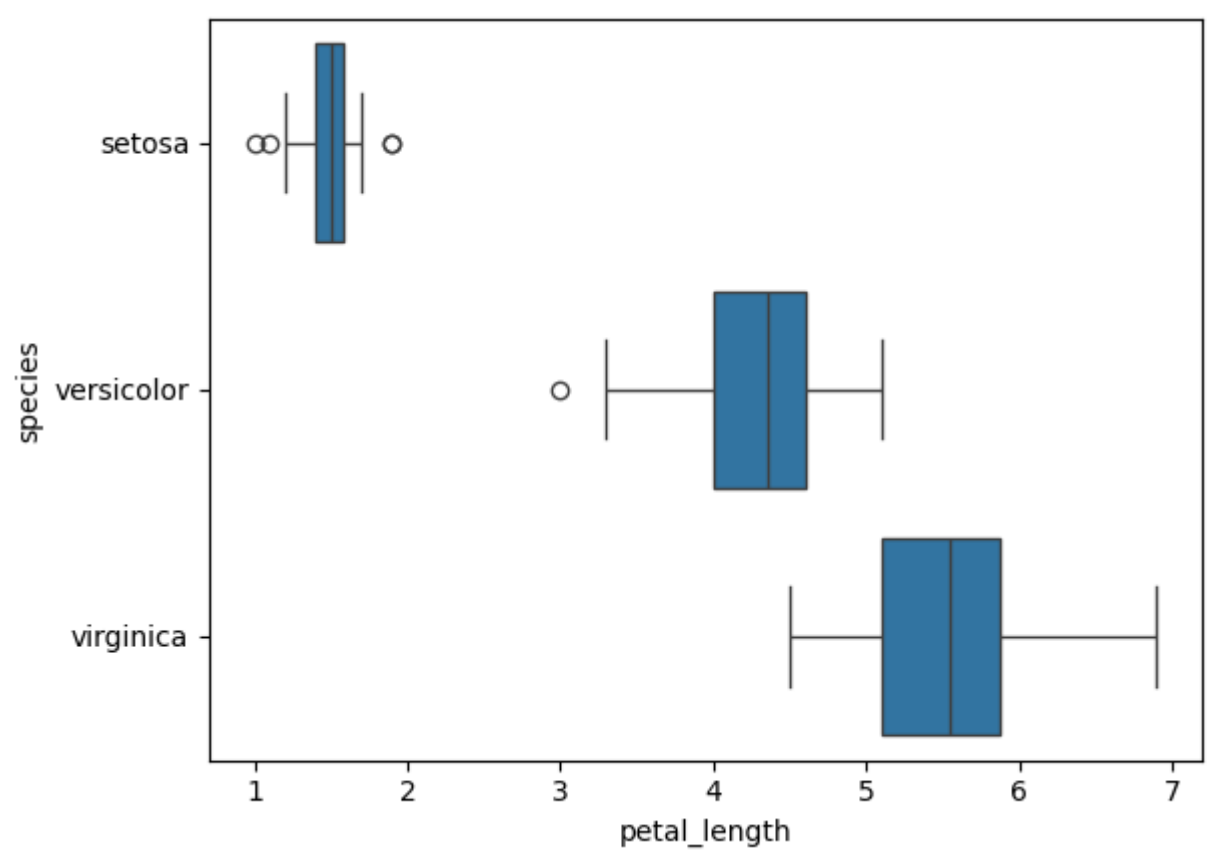
```
In [34]: sns.boxplot(x='sepal_length',y='species',data=data)
```

```
Out[34]: <Axes: xlabel='sepal_length', ylabel='species'>
```



```
In [35]: sns.boxplot(x='petal_length',y='species',data=data)
```

```
Out[35]: <Axes: xlabel='petal_length', ylabel='species'>
```

Assignment: 11

Java Code:

```
import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

    public static class TokenizerMapper extends Mapper<Object, Text, Text, IntWritable> {

        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(Object key, Text value, Context context) throws IOException,
        InterruptedException {

            StringTokenizer itr = new StringTokenizer(value.toString());
            while (itr.hasMoreTokens()) {
                word.set(itr.nextToken());
                context.write(word, one);
            }
        }
    }

    public static class IntSumReducer extends Reducer<Text, IntWritable, Text, IntWritable> {

        private IntWritable result = new IntWritable();

        public void reduce(Text key, Iterable<IntWritable> values, Context context) throws IOException,
        InterruptedException {

            int sum = 0;
```

```

        for (IntWritable val : values) {
            sum += val.get();
        }
        result.set(sum);
        context.write(key, result);
    }
}

public static void main(String[] args) throws Exception {
    if (args.length != 2) {
        System.err.println("Usage: WordCount <input> <output>");
        System.exit(1);
    }

    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "word count");
    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class);
    job.setCombinerClass(IntSumReducer.class);
    job.setReducerClass(IntSumReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

Input file:

hello hadoop hello world

Output file:

hadoop 1

hello 2

world 1

Assignment: 12

Java Code:

```
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class LogAnalysis {
    public static class LogMapper extends Mapper<Object, Text, Text, IntWritable> {
        private final static IntWritable one = new IntWritable(1);
        private Text status = new Text();

        public void map(Object key, Text value, Context context) throws IOException,
        InterruptedException {
            String line = value.toString();
            String[] parts = line.split(" ");
            for (int i = parts.length - 1; i >= 0; i--) {
                if (parts[i].matches("\\d{3}")) {
                    status.set(parts[i]);
                    context.write(status, one);
                    break;
                }
            }
        }
    }

    public static class LogReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
```

```

    public void reduce(Text key, Iterable<IntWritable> values, Context context) throws IOException,
    InterruptedException {
        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }
        context.write(key, new IntWritable(sum));
    }
}

public static void main(String[] args) throws Exception {
    if (args.length != 2) {
        System.err.println("Usage: LogAnalysis <input> <output>");
        System.exit(1);
    }

    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "Log Analysis");
    job.setJarByClass(LogAnalysis.class);
    job.setMapperClass(LogMapper.class);
    job.setCombinerClass(LogReducer.class);
    job.setReducerClass(LogReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

Input file:

```

10.223.157.186 - - [15/Jul/2009:20:50:33 -"0700] ""GET /assets/img/home""-"logo.png HTTP/1.1""
304 "-----

```

```

10.223.157.186 - - [15/Jul/2009:20:50:33 -"0700] ""GET /assets/img/dummy/primary""-news-"2.jpg
HTTP/1.1"" 304 "-----

```

10.223.157.186 - - [15/Jul/2009:20:50:33 -"0700] ""GET /assets/img/dummy/primary"-news-"1.jpg HTTP/1.1"" 304 "-----

10.223.157.186 - - [15/Jul/2009:20:50:33 -"0700] ""GET /assets/img/home"-media-block-"placeholder.jpg HTTP/1.1"" 304 "-----

10.223.157.186 - - [15/Jul/2009:20:50:33 -"0700] ""GET /assets/img/dummy/secondary"-news-"4.jpg HTTP/1.1"" 304 "-----

10.223.157.186 - - [15/Jul/2009:20:50:33 -"0700] ""GET /assets/img/loading.gif HTTP/1.1"" 304 "-----

10.223.157.186 - - [15/Jul/2009:20:50:33 -"0700] ""GET /assets/img/search"-button.gif HTTP/1.1"" 304 "-----

10.223.157.186 - - [15/Jul/2009:20:50:33 -"0700] ""GET /assets/img/dummy/secondary"-news-"3.jpg HTTP/1.1"" 304 "-----

10.223.157.186 - - [15/Jul/2009:20:50:33 -"0700] ""GET /assets/img/dummy/secondary"-news-"2.jpg HTTP/1.1"" 304 "-----

10.223.157.186 - - [15/Jul/2009:20:50:33 -"0700] ""GET /assets/img/dummy/secondary"-news-"1.jpg HTTP/1.1"" 304 "-----

10.216.113.172 - - [16/Jul/2009:02:51:31 -"0700] ""GET /assets/img/closetlabel.gif HTTP/1.1"" 200 979"-----

10.216.113.172 - - [16/Jul/2009:02:51:31 -"0700] ""GET /favicon.ico HTTP/1.1"" 404 209"-----

10.216.113.172 - - [16/Jul/2009:02:51:31 -"0700] ""GET /assets/swf/home"-media-"block.swf HTTP/1.1"" 200 123884"-----

10.216.113.172 - - [16/Jul/2009:02:51:41 -"0700] ""GET /films/district"-13 HTTP/1.1"" 301 268"-----

10.216.113.172 - - [16/Jul/2009:02:51:41 -"0700] ""GET /films/district"-13/ HTTP/1.1"" 200 12772"-----

10.216.113.172 - - [16/Jul/2009:02:51:41 -"0700] ""GET /assets/img/pdf"-icon.gif HTTP/1.1"" 200 228"-----

10.216.113.172 - - [16/Jul/2009:02:51:41 -"0700] ""GET /assets/img/dummy/momentum"-pictures-"logo.gif HTTP/1.1"" 200 3579"-----

10.216.113.172 - - [16/Jul/2009:02:51:41 -"0700] ""GET /assets/img/dummy/films/district"-13/image-"thumbs/1.jpg HTTP/1.1"" 200 9276"-----

10.216.113.172 - - [16/Jul/2009:02:51:41 -"0700] ""GET /assets/img/dummy/films/district"-13/image-"thumbs/4.jpg HTTP/1.1"" 200 10011"-----

10.216.113.172 - - [16/Jul/2009:02:51:42 -"0700] ""GET /assets/img/download"-icon.gif HTTP/1.1"" 200 71"-----

10.216.113.172 - - [16/Jul/2009:02:51:42 -"0700] ""GET /assets/img/dummy/films/district"-13/image-"thumbs/3.jpg HTTP/1.1"" 200 9040"-----

10.216.113.172 - - [16/Jul/2009:02:51:41 -"0700] ""GET /assets/img/dummy/film"-page-"media.jpg HTTP/1.1"" 200 110997"-----

10.216.113.172 - - [16/Jul/2009:02:51:42 -"0700] ""GET /assets/img/dummy/films/district"-
13/image-"thumbs/2.jpg HTTP/1.1"" 200 9626"-----

10.82.30.199 - - [16/Jul/2009:03:00:06 -"0700] ""GET / HTTP/1.1"" 200 7616"-----

10.82.30.199 - - [16/Jul/2009:03:00:06 -"0700] ""GET /assets/css/960.css HTTP/1.1"" 200 6206"-----

10.82.30.199 - - [16/Jul/2009:03:00:06 -"0700] ""GET /assets/css/reset.css HTTP/1.1"" 200 1014"----

10.82.30.199 - - [16/Jul/2009:03:00:06 -"0700] ""GET /assets/js/lowpro.js HTTP/1.1"" 200 10469"---

10.82.30.199 - - [16/Jul/2009:03:00:06 -"0700] ""GET /assets/css/the-"-associates.css HTTP/1.1""
200 15779"-----

10.82.30.199 - - [16/Jul/2009:03:00:06 -"0700] ""GET /assets/js/the-"-associates.js HTTP/1.1"" 200
4492"-----

10.82.30.199 - - [16/Jul/2009:03:00:06 -"0700] ""GET /assets/js/lightbox.js HTTP/1.1"" 200 25960"--

10.82.30.199 - - [16/Jul/2009:03:00:07 -"0700] ""GET /assets/img/home-"-logo.png HTTP/1.1"" 200
3892"-----

10.82.30.199 - - [16/Jul/2009:03:00:07 -"0700] ""GET /assets/img/dummy/secondary"-news-"3.jpg
HTTP/1.1"" 200 5604"-----

10.82.30.199 - - [16/Jul/2009:03:00:07 -"0700] ""GET /assets/img/dummy/primary"-news-"1.jpg
HTTP/1.1"" 200 10556"-----

10.82.30.199 - - [16/Jul/2009:03:00:07 -"0700] ""GET /assets/img/dummy/secondary"-news-"4.jpg
HTTP/1.1"" 200 5766"-----

10.82.30.199 - - [16/Jul/2009:03:00:07 -"0700] ""GET /assets/img/dummy/primary"-news-"2.jpg
HTTP/1.1"" 200 9925"-----

10.82.30.199 - - [16/Jul/2009:03:00:07 -"0700] ""GET /assets/img/dummy/secondary"-news-"1.jpg
HTTP/1.1"" 200 5766"-----

Output file:

200	1110
206	28
301	12
302	4
304	108
401	2
404	27
405	2
416	1

Assignment: 13

Java Code:

```
package com.org.vasanth.weather;

import java.io.IOException;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.StringTokenizer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapred.FileInputFormat;
import org.apache.hadoop.mapred.FileOutputFormat;
import org.apache.hadoop.mapred.JobClient;
import org.apache.hadoop.mapred.JobConf;
import org.apache.hadoop.mapred.KeyValueTextInputFormat;
import org.apache.hadoop.mapred.MapReduceBase;
import org.apache.hadoop.mapred.Mapper;
import org.apache.hadoop.mapred.OutputCollector;
import org.apache.hadoop.mapred.Reducer;
import org.apache.hadoop.mapred.Reporter;
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;

public class Weather extends Configured implements Tool {

    final long DEFAULT_SPLIT_SIZE = 128 * 1024 * 1024;

    public static class MapClass extends MapReduceBase implements Mapper<LongWritable, Text, Text, Text> {

        private Text word = new Text();
        private Text values = new Text();
```



```

public void map(LongWritable key, Text value, OutputCollector<Text, Text> output, Reporter
reporter) throws IOException {

    String line = value.toString();

    StringTokenizer itr = new StringTokenizer(line);

    int counter = 0;

    String key_out = null;

    String value_str = null;

    boolean skip = false;

    loop:while (itr.hasMoreTokens() && counter<13) {

        String str = itr.nextToken();

        switch (counter) {

        case 0:

            key_out = str;

            if(str.contains("STN")){

                skip = true;

                break loop;

            }else{

                break;

            }

        case 2:

            int hour = Integer.valueOf(str.substring(str.lastIndexOf("_")+1, str.length()));

            str = str.substring(4,str.lastIndexOf("_")-2);

            if(hour>4 && hour<=10){

                str = str.concat("_section1");

            }else if(hour>10 && hour<=16){

                str = str.concat("_section2");

            }else if(hour>16 && hour<=22){

                str = str.concat("_section3");

            }else{

                str = str.concat("_section4");

            }

            key_out = key_out.concat("_").concat(str);

            break;

```

case 3:

```
if(str.equals("9999.9")){  
    skip = true;  
    break loop;  
}else{  
    value_str = str.concat(" ");  
    break;  
}
```

case 4:

```
if(str.equals("9999.9")){  
    skip = true;  
    break loop;  
}else{  
    value_str = value_str.concat(str).concat(" ");  
    break;  
}
```

case 12:

```
if(str.equals("999.9")){  
    skip = true;  
    break loop;  
}else{  
    value_str = value_str.concat(str).concat(" ");  
    break;  
}
```

default:

```
    break;  
}  
counter++;
```

```
}
```

```
if(!skip){
```

```
    word.set(key_out);
```

```
    values.set(value_str);
```

```

        output.collect(word, values);
    }
}

}

public static class MapClassForJob2 extends MapReduceBase implements Mapper<Text, Text,
Text, Text> {
    private Text key_text = new Text();
    private Text value_text = new Text();

    public void map(Text key, Text value, OutputCollector<Text, Text> output, Reporter reporter)
throws IOException {
        String str = key.toString();
        String station = str.substring(str.lastIndexOf("_")+1, str.length());
        str = str.substring(0, str.lastIndexOf("_"));
        key_text.set(str);
        StringTokenizer itr = new StringTokenizer(value.toString());
        String str_out = station.concat("<");
        while (itr.hasMoreTokens()) {
            String nextToken = itr.nextToken(" ");
            str_out = str_out.concat(nextToken);
            str_out = ((itr.hasMoreTokens()) ? str_out.concat(",") : str_out.concat(">"));
        }
        value_text.set(str_out);
        output.collect(key_text, value_text);
    }
}

public static class Reduce extends MapReduceBase implements Reducer<Text, Text, Text, Text> {
    private Text value_out_text = new Text();

    public void reduce(Text key, Iterator<Text> values, OutputCollector<Text, Text> output,
Reporter reporter) throws IOException {
        double sum_temp = 0;
        double sum_dew = 0;
        double sum_wind = 0;
        int count = 0;

```

```

while (values.hasNext()) {
    String str = values.next().toString();
    StringTokenizer itr = new StringTokenizer(str);
    int count_vector = 0;
    while (itr.hasMoreTokens()) {
        String nextToken = itr.nextToken(" ");
        if(count_vector==0){
            sum_temp += Double.valueOf(nextToken);
        }
        if(count_vector==1){
            sum_dew += Double.valueOf(nextToken);
        }
        if(count_vector==2){
            sum_wind += Double.valueOf(nextToken);
        }
        count_vector++;
    }
    count++;
}

double avg_tmp = sum_temp / count;
double avg_dew = sum_dew / count;
double avg_wind = sum_wind / count;

String value_out = String.valueOf(avg_tmp).concat("
").concat(String.valueOf(avg_dew)).concat(" ").concat(String.valueOf(avg_wind));

value_out_text.set(value_out);
output.collect(key, value_out_text);
}
}

public static class ReduceForJob2 extends MapReduceBase implements Reducer<Text, Text, Text,
Text> {

    private Text value_out_text = new Text();

    public void reduce(Text key, Iterator<Text> values, OutputCollector<Text, Text> output,
Reporter reporter) throws IOException {

```

```

        String value_out = "";
        while (values.hasNext()) {
            value_out = value_out.concat(values.next().toString()).concat(" ");
        }
        value_out_text.set(value_out);
        output.collect(key, value_out_text);
    }
}

static int printUsage() {
    System.out.println("weather [-m <maps>] [-r <reduces>] <job_1 input> <job_1 output> <job_2 output>");
    ToolRunner.printGenericCommandUsage(System.out);
    return -1;
}

public int run(String[] args) throws Exception {
    Configuration config = getConf();
    JobConf conf = new JobConf(config, Weather.class);
    conf.setJobName("Weather Job1");
    conf.setOutputKeyClass(Text.class);
    conf.setOutputValueClass(Text.class);
    conf.setMapOutputKeyClass(Text.class);
    conf.setMapOutputValueClass(Text.class);
    conf.setMapperClass(MapClass.class);
    conf.setReducerClass(Reduce.class);
    List<String> other_args = new ArrayList<String>();
    for(int i=0; i < args.length; ++i) {
        try {
            if ("-m".equals(args[i])) {
                conf.setNumMapTasks(Integer.parseInt(args[++i]));
            } else if ("-r".equals(args[i])) {
                conf.setNumReduceTasks(Integer.parseInt(args[++i]));
            } else {
                other_args.add(args[i]);
            }
        }
    }
}

```

```

    }
    } catch (NumberFormatException except) {
        System.out.println("ERROR: Integer expected instead of " + args[i]);
        return printUsage();
    } catch (ArrayIndexOutOfBoundsException except) {
        System.out.println("ERROR: Required parameter missing from " + args[i-1]);
        return printUsage();
    }
}

FileInputFormat.setInputPaths(conf, other_args.get(0));
FileOutputFormat.setOutputPath(conf, new Path(other_args.get(1)));
JobClient.runJob(conf);
JobConf conf2 = new JobConf(config, Weather.class);
conf2.setJobName("Weather Job 2");
conf2.setOutputKeyClass(Text.class);
conf2.setOutputValueClass(Text.class);
conf2.setInputFormat(KeyValueTextInputFormat.class);
conf2.setMapOutputKeyClass(Text.class);
conf2.setMapOutputValueClass(Text.class);
conf2.setMapperClass(MapClassForJob2.class);
conf2.setReducerClass(ReduceForJob2.class);
FileInputFormat.setInputPaths(conf2, new Path(other_args.get(1)));
FileOutputFormat.setOutputPath(conf2, new Path(other_args.get(2)));
JobClient.runJob(conf2);
return 0;
}

public static void main(String[] args) throws Exception {
    int res = ToolRunner.run(new Configuration(), new Weather(), args);
    System.exit(res);
}
}

```

Input file:

690190	13910	20060201_0	51.75	33.0	24	1006.3	24	943.9	24	15.0	24	10.7	24	22.0	28.9
0.00I 999.9 000000															
690190	13910	20060201_1	54.74	33.0	24	1006.3	24	943.9	24	15.0	24	10.7	24	22.0	28.9
0.00I 999.9 000000															
690190	13910	20060201_2	50.59	33.0	24	1006.3	24	943.9	24	15.0	24	10.7	24	22.0	28.9
0.00I 999.9 000000															
690190	13910	20060201_3	51.67	33.0	24	1006.3	24	943.9	24	15.0	24	10.7	24	22.0	28.9
0.00I 999.9 000000															
690190	13910	20060201_4	65.67	33.0	24	1006.3	24	943.9	24	15.0	24	10.7	24	22.0	28.9
0.00I 999.9 000000															
690190	13910	20060201_5	55.37	33.0	24	1006.3	24	943.9	24	15.0	24	10.7	24	22.0	28.9
0.00I 999.9 000000															
690190	13910	20060201_6	49.26	33.0	24	1006.3	24	943.9	24	15.0	24	10.7	24	22.0	28.9
0.00I 999.9 000000															
690190	13910	20060201_7	55.44	33.0	24	1006.3	24	943.9	24	15.0	24	10.7	24	22.0	28.9
0.00I 999.9 000000															
690190	13910	20060201_8	64.05	33.0	24	1006.3	24	943.9	24	15.0	24	10.7	24	22.0	28.9
0.00I 999.9 000000															
690190	13910	20060201_9	68.77	33.0	24	1006.3	24	943.9	24	15.0	24	10.7	24	22.0	28.9
0.00I 999.9 000000															
690190	13910	20060201_10	48.93	33.0	24	1006.3	24	943.9	24	15.0	24	10.7	24	22.0	28.9
0.00I 999.9 000000															
690190	13910	20060201_11	65.37	33.0	24	1006.3	24	943.9	24	15.0	24	10.7	24	22.0	28.9
0.00I 999.9 000000															
690190	13910	20060201_12	69.45	33.0	24	1006.3	24	943.9	24	15.0	24	10.7	24	22.0	28.9
0.00I 999.9 000000															
690190	13910	20060201_13	52.91	33.0	24	1006.3	24	943.9	24	15.0	24	10.7	24	22.0	28.9
0.00I 999.9 000000															
690190	13910	20060201_14	53.69	33.0	24	1006.3	24	943.9	24	15.0	24	10.7	24	22.0	28.9
0.00I 999.9 000000															
690190	13910	20060201_15	53.30	33.0	24	1006.3	24	943.9	24	15.0	24	10.7	24	22.0	28.9
0.00I 999.9 000000															
690190	13910	20060201_16	66.17	33.0	24	1006.3	24	943.9	24	15.0	24	10.7	24	22.0	28.9
0.00I 999.9 000000															
690190	13910	20060201_17	53.83	33.0	24	1006.3	24	943.9	24	15.0	24	10.7	24	22.0	28.9
0.00I 999.9 000000															
690190	13910	20060201_18	50.54	33.0	24	1006.3	24	943.9	24	15.0	24	10.7	24	22.0	28.9
0.00I 999.9 000000															

690190	13910	20060201_19	50.27	33.0 24	1006.3 24	943.9 24	15.0 24	10.7 24	22.0	28.9
0.00I 999.9 000000										
690190	13910	20060201_20	59.08	33.0 24	1006.3 24	943.9 24	15.0 24	10.7 24	22.0	28.9
0.00I 999.9 000000										
690190	13910	20060201_21	53.05	33.0 24	1006.3 24	943.9 24	15.0 24	10.7 24	22.0	28.9
0.00I 999.9 000000										
690190	13910	20060201_22	57.97	33.0 24	1006.3 24	943.9 24	15.0 24	10.7 24	22.0	28.9
0.00I 999.9 000000										
690190	13910	20060201_23	48.23	33.0 24	1006.3 24	943.9 24	15.0 24	10.7 24	22.0	28.9
0.00I 999.9 000000										
690190	13910	20060202_0	47.16	28.5 24	1003.2 24	940.6 24	15.0 24	5.7 24	12.0	999.9
0.00I 999.9 000000										
690190	13910	20060202_1	69.72	28.5 24	1003.2 24	940.6 24	15.0 24	5.7 24	12.0	999.9
0.00I 999.9 000000										
690190	13910	20060202_2	62.71	28.5 24	1003.2 24	940.6 24	15.0 24	5.7 24	12.0	999.9
0.00I 999.9 000000										
690190	13910	20060202_3	46.34	28.5 24	1003.2 24	940.6 24	15.0 24	5.7 24	12.0	999.9
0.00I 999.9 000000										
690190	13910	20060202_4	53.15	28.5 24	1003.2 24	940.6 24	15.0 24	5.7 24	12.0	999.9
0.00I 999.9 000000										
690190	13910	20060202_5	64.59	28.5 24	1003.2 24	940.6 24	15.0 24	5.7 24	12.0	999.9
0.00I 999.9 000000										
690190	13910	20060202_6	58.26	28.5 24	1003.2 24	940.6 24	15.0 24	5.7 24	12.0	999.9
0.00I 999.9 000000										
690190	13910	20060202_7	53.27	28.5 24	1003.2 24	940.6 24	15.0 24	5.7 24	12.0	999.9
0.00I 999.9 000000										
690190	13910	20060202_8	43.68	28.5 24	1003.2 24	940.6 24	15.0 24	5.7 24	12.0	999.9
0.00I 999.9 000000										
690190	13910	20060202_9	65.70	28.5 24	1003.2 24	940.6 24	15.0 24	5.7 24	12.0	999.9
0.00I 999.9 000000										
690190	13910	20060202_10	66.27	28.5 24	1003.2 24	940.6 24	15.0 24	5.7 24	12.0	999.9
0.00I 999.9 000000										
690190	13910	20060202_11	53.05	28.5 24	1003.2 24	940.6 24	15.0 24	5.7 24	12.0	999.9
0.00I 999.9 000000										
690190	13910	20060202_12	68.45	28.5 24	1003.2 24	940.6 24	15.0 24	5.7 24	12.0	999.9
0.00I 999.9 000000										
690190	13910	20060202_13	49.03	28.5 24	1003.2 24	940.6 24	15.0 24	5.7 24	12.0	999.9
0.00I 999.9 000000										
690190	13910	20060202_14	66.59	28.5 24	1003.2 24	940.6 24	15.0 24	5.7 24	12.0	999.9
0.00I 999.9 000000										

690190 13910 20060202_15	63.12	28.5 24	1003.2 24	940.6 24	15.0 24	5.7 24	12.0	999.9	0.00I 999.9 000000
690190 13910 20060202_16	49.13	28.5 24	1003.2 24	940.6 24	15.0 24	5.7 24	12.0	999.9	0.00I 999.9 000000
690190 13910 20060202_17	62.85	28.5 24	1003.2 24	940.6 24	15.0 24	5.7 24	12.0	999.9	0.00I 999.9 000000
690190 13910 20060202_18	64.67	28.5 24	1003.2 24	940.6 24	15.0 24	5.7 24	12.0	999.9	0.00I 999.9 000000
690190 13910 20060202_19	55.73	28.5 24	1003.2 24	940.6 24	15.0 24	5.7 24	12.0	999.9	0.00I 999.9 000000
690190 13910 20060202_20	56.42	28.5 24	1003.2 24	940.6 24	15.0 24	5.7 24	12.0	999.9	0.00I 999.9 000000
690190 13910 20060202_21	53.83	28.5 24	1003.2 24	940.6 24	15.0 24	5.7 24	12.0	999.9	0.00I 999.9 000000
690190 13910 20060202_22	45.14	28.5 24	1003.2 24	940.6 24	15.0 24	5.7 24	12.0	999.9	0.00I 999.9 000000
690190 13910 20060202_23	68.18	28.5 24	1003.2 24	940.6 24	15.0 24	5.7 24	12.0	999.9	0.00I 999.9 000000
690190 13910 20060203_0	48.41	26.5 24	1008.7 24	946.0 24	15.0 24	9.6 24	17.1	22.0	0.00I 999.9 000000
690190 13910 20060203_1	55.12	26.5 24	1008.7 24	946.0 24	15.0 24	9.6 24	17.1	22.0	0.00I 999.9 000000
690190 13910 20060203_2	46.48	26.5 24	1008.7 24	946.0 24	15.0 24	9.6 24	17.1	22.0	0.00I 999.9 000000
690190 13910 20060203_3	54.99	26.5 24	1008.7 24	946.0 24	15.0 24	9.6 24	17.1	22.0	0.00I 999.9 000000
690190 13910 20060203_4	50.62	26.5 24	1008.7 24	946.0 24	15.0 24	9.6 24	17.1	22.0	0.00I 999.9 000000
690190 13910 20060203_5	55.81	26.5 24	1008.7 24	946.0 24	15.0 24	9.6 24	17.1	22.0	0.00I 999.9 000000
690190 13910 20060203_6	59.28	26.5 24	1008.7 24	946.0 24	15.0 24	9.6 24	17.1	22.0	0.00I 999.9 000000

Output file:

690190_02_section1	53.87166666666665	25.900000000000006	7.774999999999995
690190_02_section2	54.76125	25.900000000000006	7.774999999999998
690190_02_section3	53.25041666666666	25.900000000000006	7.774999999999998
690190_02_section4	52.44708333333333	25.900000000000002	7.774999999999998